Ahmed Mohamad Bakri
Matheus Marques de Miranda

Sistemas de identidade descentralizada auto-soberana: Uma prova de conceito para aplicação em condomínios

Brasil

2024

Ahmed Mohamad Bakri
Matheus Marques de Miranda

Sistemas de identidade descentralizada auto-soberana: Uma prova de conceito para aplicação em condomínios

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac – Santo Amaro

Bacharelado em Ciência da Computação

Orientador: Prof. Dr. Eduardo Takeo Ueda

Brasil

2024

Dedicamos este trabalho às nossas famílias,
que sempre nos apoiaram e nos deram força para seguir em frente.
E a todos que contribuíram direta ou indiretamente para nossa formação.

Agradecimentos

À todos professores do curso de Ciência da Computação do Centro Universitário Senac Santo Amaro, pelos ensinamentos durante toda a graduação. Sem eles, não teríamos chegado até aqui. Aos nosso colegas de turma, por todo apoio e companheirismo ao longo desta jornada.

Em especial agradecemos ao professor Dr. Eduardo Takeo Ueda e ao professor Thyago Quintas, que nos orientaram durante todo desenvolvimento deste trabalho, com paciência, dedicação, conhecimento e longas conversas.

Por fim agradecemos à comunidade do Hyperledger Foundation e OpenWallet Foundation, que foram fundamentais no entendimento dos conceitos abordados neste trabalho.

Resumo

Esta monografia tem como objetivo apresentar uma prova de conceito sobre sistemas de identidade descentralizada auto soberana aplicados em condomínios. A identidade auto soberana é um conceito que permite aos usuários donos de identidades digitais terem controle total sobre suas próprias informações pessoais. Aliada a tecnologias de blockchain, criptografia e identificadores descentralizados, que juntos garantem a integridade, autenticidade e privacidade dos dados. Esse conceito faz com que os indivíduos possam manter seus dados em um ambiente de sua escolha, como seu próprio dispositivo móvel ou computador. Durante a leitura do trabalho, será feito uma revisão da literatura sobre temas relacionados à identidade auto soberana, para que possamos entender melhor suas aplicações existentes. E após a revisão, será apresentado a prova de conceito de um sistema de identidade auto soberana para liberação de acesso em condomínios.

Palavras-chaves: Blockchain. Identificadores descentralizados. Identidade auto soberana. Condomínios.

Abstract

This monograph aims to present a proof of concept on decentralized self-sovereign identity systems applied to condominiums. Self-sovereign identity is a concept that enables individuals to have full control over their digital identities and personal information. Coupled with blockchain technology, cryptography, and decentralized identifiers, it ensures data integrity, authenticity, and privacy. This approach allows individuals to store their data in an environment of their choice, such as their personal mobile device or computer. The study includes a literature review on topics related to self-sovereign identity to provide a deeper understanding of its existing applications. Following the review, the proof of concept for a self-sovereign identity system designed to manage access control in condominiums is presented.

Key-words: Blockchain. Self-sovereign identity. Descentralized identifiers. Condominiums.

Lista de ilustrações

Figura 1 – Criptografia Simétrica 25
Figura 2 – Criptografia Assimétrica 25
Figura 3 – Exemplo de Hash criptográfico 26
Figura 4 – Exemplo de Assinatura Digital 27
Figura 5 – Tipos de entidades e seus atributos 30
Figura 6 – Exemplo de Estrutura dos sistemas de identidade 30
Figura 7 – Exemplo de Blockchain 36
Figura 8 – Exemplo simples de did 38
Figura 9 – Componentes DID 39
Figura 10 – Exemplo simples de um documento did. 39
Figura 11 – Arquitetura DID. 40
Figura 12 – Exemplo de Estrutura de uma Credencial Verificável 42
Figura 13 – Ciclo de Vida de uma Credencial Verificável 43
Figura 14 – Exemplo de Uso de *Anoncred* 46
Figura 15 – Arquitetura do Acapy 54
Figura 16 – Arquitetura da aplicação 57
Figura 17 – Dashboard do Condomínio 58
Figura 18 – Criação de Schema (Condomínio) 58
Figura 19 – Dashboard do Residente 59
Figura 20 – Ofertas de Credencial (Residente) 59
Figura 21 – Inicialização do agente ACA-Py 60
Figura 22 – Interface da Von-Network 61
Figura 23 – Registros dentro da Blockchain 61
Figura 24 – Diagrama de conexão entre agentes 62
Figura 25 – Convite de conexão (Condomínio) 62
Figura 26 – Conexões (Condomínio) 63
Figura 27 – Emissão de credencial 63
Figura 28 – Oferta de credencial (Condomínio) 64
Figura 29 – Verificação de credencial 65
Figura 30 – Prova de residência (Condomínio) 65
Figura 31 – Apresentar prova (Residente) 66
Figura 32 – *Pipeline* de emissão de credenciais 67
Figura 33 – *Pipeline* de verificação de credenciais 69

Lista de tabelas

Tabela 1 – Princípios de sistemas de identidade descentralizada e seus exemplos . 32
Tabela 2 – Elementos de uma Credencial Verificável e suas descrições 41
Tabela 3 – Características das Credenciais Verificáveis 42
Tabela 4 – Domínios de Aplicação de Credenciais Verificáveis 43
Tabela 5 – Tipos de Provas de Conhecimento Zero 45

Lista de abreviaturas e siglas

ACA-Py	Aries Cloud Agent Python
API	Application programing interface - Interface de programação de aplicações
DID	Identificador descentralizado
DLT	Distributed ledger technology - Tecnologia de Registro distribuído
NFC	Near Field Communication - Comunicação de campo próximo
REST	Representational State Transfer - Transferência de estado representacional
SSI	Self sovereign identity - Identidade auto-soberana
URI	Uniform Resource Identifier - Identificador Uniforme de Recursos
URL	Uniform Resource Locator - Localizador uniforme de recursos
VC	Verifiable Credentials - Credenciais Verificáveis
W3C	World Wide Web Consortium - Consórcio da World Wide Web
ZKP	Zero Knowledge Proofs - Provas de conhecimento zero

Sumário

Sumário 17

1 INTRODUÇÃO 21
1.1 Contexto 21

2 SEGURANÇA DE DADOS 23
2.1 Síntese 23
2.2 Definição de segurança de dados 24
2.3 Fundamentos de Criptografia 24
2.3.1 Criptografia Simétrica e Assimétrica 24
2.3.2 Funções Hash Criptográficas 26
2.3.3 Assinaturas Digitais 27

3 IDENTIDADE DIGITAL 29
3.1 Síntese 29
3.2 Definição e tipos de entidades 29
3.3 Sistemas de Identidade 30
3.4 Identidades Digital Autos-soberanas 32

4 TECNOLOGIAS PARA IDENTIDADES DIGITAIS AUTO-SOBERANAS 35
4.1 Síntese 35
4.2 Blockchain 35
4.2.1 Blockchain Permissionada 36
4.3 Identificadores Descentralizados 37
4.3.1 Formato DID 37
4.3.2 URL DID 38
4.3.3 Documento DID 39
4.3.4 Arquitetura DID 40
4.4 Credenciais Verificáveis 41
4.4.1 Definição e Estrutura 41
4.4.2 Processo de Emissão, Apresentação e Verificação 41
4.4.3 Vantagens das Credenciais Verificáveis 42
4.4.4 Aplicações das Credenciais Verificáveis 43
4.5 Provas de Conhecimento Zero 44
4.5.1 Definição e Princípios Fundamentais 44
4.5.2 Provas de Conhecimento Zero em Credenciais Verificáveis 44

4.5.3 Tipos de Provas de Conhecimento Zero 45
4.5.4 Anoncreds: Uma Aplicação Prática das ZKPs 45

5 PLATAFORMAS DE IDENTIDADE AUTO-SOBERANA 47
5.1 Síntese 47
5.2 Hyperledger Foundation 47
5.2.1 Projetos Hyperledger 48
5.3 Comparação com Outras Plataformas de SSI 48
5.3.1 Sovrin 48
5.3.2 uPort/Veramo 49

6 SEGURANÇA DE CONDOMÍNIOS 51

7 IMPLEMENTAÇÃO DA PROVA DE CONCEITO 53
7.1 Síntese 53
7.2 Tecnologias usadas 53
7.2.1 Python 53
7.2.1.1 ACA-Py 53
7.2.1.1.1 OpenAPI 55
7.2.2 React 55
7.2.3 Docker 55
7.3 Tecnologia Blockchain 56
7.3.1 Indy-node 56
7.3.1.1 VON-network 56
7.4 Aplicação 57
7.4.1 Arquitetura 57
7.4.1.1 Frontend 57
7.4.1.2 Backend 60
7.4.1.3 Blockchain 60
7.4.2 Fluxo de uso da aplicação 62
7.4.2.1 Fluxo da conexão entre agentes 62
7.4.2.2 Fluxo da emissão de credencial 63
7.4.2.3 Fluxo da verificação de credencial 64
7.4.3 Tabelas das requisições 66
7.4.3.1 Tabela de emissão de credencial 66
7.4.3.2 Tabela de verificação de credencial (Prova) 68

8 RESULTADOS E DISCUSSÃO 71

9 CONCLUSÃO 73
9.1 Trabalhos futuros 73

REFERÊNCIAS 75

Apêndices 80

A – PREPARAÇÃO DO AMBIENTE 81
A.1 Pré requisitos 81

B – EXECUTAÇÃO DO SISTEMA 83
B.1 Frontend 83
B.2 Von network 83
B.3 Inicialização dos agentes 84

1 Introdução

1.1 Contexto

Nos últimos anos, a necessidade de preservar a privacidade e de garantir a identificação digital levou a evoluções tecnológicas na maneira como a gestão de identidades é realizada. Temos como exemplo plataformas que atualmente fazem a gestão de identidade e de acessos inteiramente pelo celular (Access Run, 2019).

Segundo o Fórum Econômico Mundial, "é necessário que indivíduos, entidades legais e dispositivos possuam uma identidade verificável e confiável para que possam interagir e realizar transações com segurança."(World Economic Forum, 2018). Sendo assim, vemos a importância e a necessidade de desenvolver soluções que permitam o controle dessas identidades de maneira segura e eficiente. Em 2019, o W3C publicou a recomendação do DID-Core (SPORNY et al., 2022), que representa o primeiro esforço de padronização relacionado aos *Decentralized Identifiers* (DIDs). Este padrão aborda as especificações técnicas para a criação, resolução e utilização de identificadores descentralizados, estabelecendo uma base sólida para autenticação e autorização no contexto de identidades digitais descentralizadas (ALZAHRANI; BANDER, 2020). DID é de extrema importância, pois possibilita que indivíduos tenham controle total sobre suas próprias identidades digitais sem a necessidade de intermediários centralizados.

No entanto DID's não operam isoladamente. O conceito de **identidade auto soberana** ou ***Self-Sovereign Identity*** **(SSI)** é o que une os identificadores descentralizados com mecanismos de controle e gestão de identidade que colocam o usuário no centro do processo, permitindo a criação, armazenamento e compartilhamento de suas credenciais de forma segura e independente de autoridades centralizadas. Segundo Okta, é a maneira do usuário manter total controle sobre sua identidade digital, ou seja, o usuário é quem decide quem vê quais informações e quando (Okta, 2024).

Colocar o usuário como o controlador da sua própria identidade não é a abordagem convencional para gestão de identidades, geralmente terceiros, como empresas, organizações ou governos atuam como intermediários na detenção e gerenciamento das identidades. Nesse modelo, estamos expostos a alguns riscos como vazamento de dados, violações de privacidade e ataques cibernéticos. Em contraste, a abordagem de SSI promove um paradigma descentralizado e inovador, utilizando conceitos da criptografia, como credenciais verificáveis, identificadores descentralizados e *blockchain* que permitem a criação de um registro seguro das credenciais do usuário.

Um exemplo da unificação de SSI e *blockchain* é o *Hyperledger Indy*, uma plataforma especificamente projetada para suportar identidades descentralizadas. O *Hyperledger Indy* utiliza a tecnologia *blockchain* para fornecer uma infraestrutura robusta e confiável para o gerenciamento de DIDs e credenciais verificáveis. Essa plataforma permite que os usuários emitam, armazenem e verifiquem suas credenciais digitais de forma segura, sem a necessidade de intermediários tradicionais (FOUNDATION, 2024d).

O objetivo deste trabalho de conclusão de curso é construir uma prova de conceito, implementando o conceito de DID's e SSI em uma aplicação web capaz de emitir e gerenciar credenciais descentralizadas especificamente em condomínios residenciais. Para isso utilizamos a ferramenta *Hyperledger Aries* que permite emitir e gerenciar essas credenciais de forma segura.

2 Segurança de Dados

2.1 Síntese

Neste capítulo, exploramos a importância fundamental da segurança de dados no contexto atual, destacando a necessidade de proteger informações contra cibercriminosos, ataques internos e erros humanos. Discutimos a definição de segurança de dados, enfatizando a preservação da confidencialidade, integridade e disponibilidade das informações. Abordamos diversas técnicas de proteção, como criptografia, gerenciamento de chaves e controle de acesso.

Adicionalmente, analisamos os fundamentos da criptografia, mostrando dois tipos de criptografia, simétrica e assimétrica e diferenciando o uso de chaves e a segurança da troca de informação dos dois tipos. Também discutimos o papel das funções hash criptográficas na garantia da integridade dos dados, transformando entradas de tamanho variável em saídas de tamanho fixo e resistentes a colisões.

Por fim, examinamos as assinaturas digitais como um complemento essencial às funções hash, assegurando a autenticidade e autoria dos dados. As assinaturas digitais utilizam criptografia assimétrica para permitir que remetentes provem a autenticidade de suas mensagens ou documentos, garantindo que os receptores possam verificar a integridade e a origem das informações recebidas. Este capítulo estabelece uma base sólida para entender as práticas e tecnologias de segurança de dados, fundamentais para a implementação de sistemas de identidade descentralizada que serão abordados nos capítulos seguintes.

2.2 Definição de segurança de dados

Dados são um dos ativos mais importantes de uma organização e também para indivíduos. Sua proteção é essencial contra cibercriminosos, ataques internos e também contra o erro humano. Portanto a **segurança de dados** é essencial nos dias atuais com a transformação digital, e é importante que a definição desse conceito esteja bastante clara antes de prosseguirmos. De acordo com a Oracle, "A segurança de dados refere-se às medidas de proteção empregadas para proteger os dados contra acesso não aprovado e para preservar a confidencialidade, integridade e disponibilidade dos dados."(ORACLE, 2024). É possível proteger dados com técnicas como criptografia, gerenciamento de chaves e controle de acesso aos dados para usuários sem privilégios.

2.3 Fundamentos de Criptografia

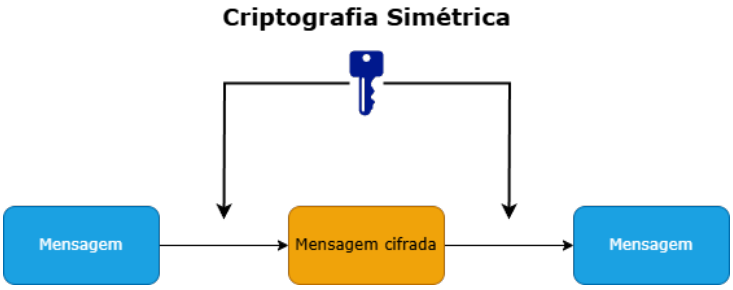
A criptografia consiste no estudo de técnicas destinadas a garantir a confidencialidade, integridade e autenticidade das informações (STALLINGS, 2022). Ela é um dos princípios básicos na área de Segurança de dados e envolve dois conceitos importantes, sendo eles:

2.3.1 Criptografia Simétrica e Assimétrica

Uma chave representa um valor, geralmente uma sequência binária, utilizado em algoritmos de criptografia para transformar dados legíveis em dados criptografados. E também é utilizada para fazer o caminho inverso, onde temos dados cifrados e queremos recuperar os dados legíveis. Sabendo disso podemos introduzir os tipos de criptografia que utiliza diferentes tipos de chaves.

- **Criptografia Simétrica**: Utiliza a mesma chave para criptografar e descriptografar mensagens. Embora seja eficiente, o desafio está na distribuição segura das chaves entre as partes (KATZ; LINDELL, 2020).

Figura 1 – Criptografia Simétrica

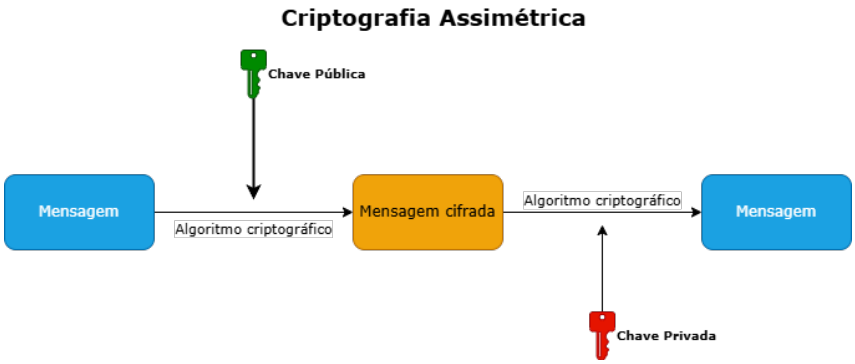


Fonte: Autoral

A Figura 1 apresenta um diagrama onde uma mensagem é criptografada e depois decifrada utilizando uma mesma chave criptográfica.

- **Criptografia Assimétrica**: Utiliza um par de chaves, uma pública e outra privada. A chave pública pode ser acessada por todos, enquanto a chave privada deve ser mantida em segredo pelo proprietário. Esse modelo facilita a troca segura de informações sem a necessidade de compartilhar chaves secretas previamente (DIFFIE; HELLMAN, 1976).

Figura 2 – Criptografia Assimétrica



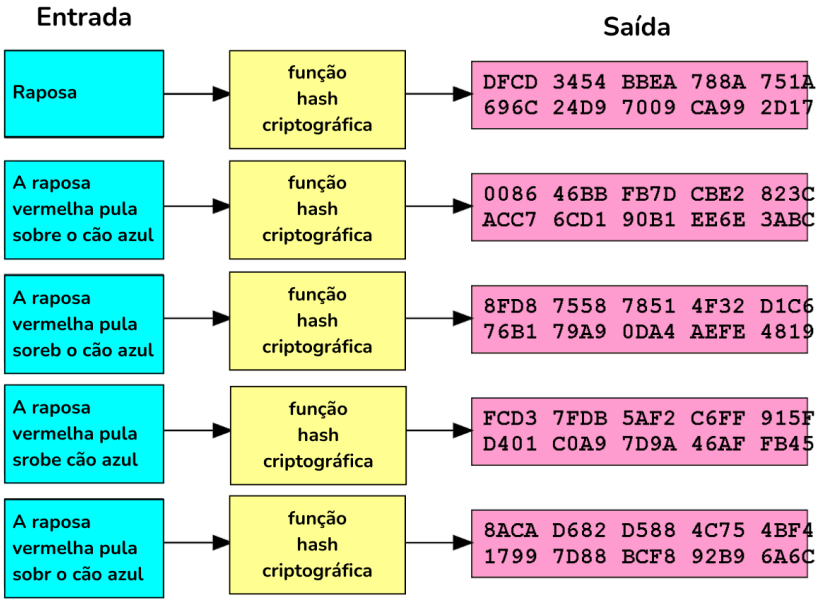
Fonte: Autoral

A Figura 2 apresenta um diagrama uma mensagem é criptografada utilizando uma chave pública de acesso a todos e depois ela é decifrada utilizando uma chave privada que apenas o proprietário da mesma possui o acesso.

2.3.2 Funções Hash Criptográficas

As funções hash criptográficas são algoritmos que transformam uma entrada de dados de tamanho variável em uma saída de tamanho fixo, conhecida como hash. Essas funções são unidirecionais e resistentes a colisões, sendo fundamentais para garantir a integridade dos dados em sistemas como o blockchain (STALLINGS, 2022).

Figura 3 – Exemplo de Hash criptográfico



Fonte: Adaptado de (Bit2Me Academy, 2018)

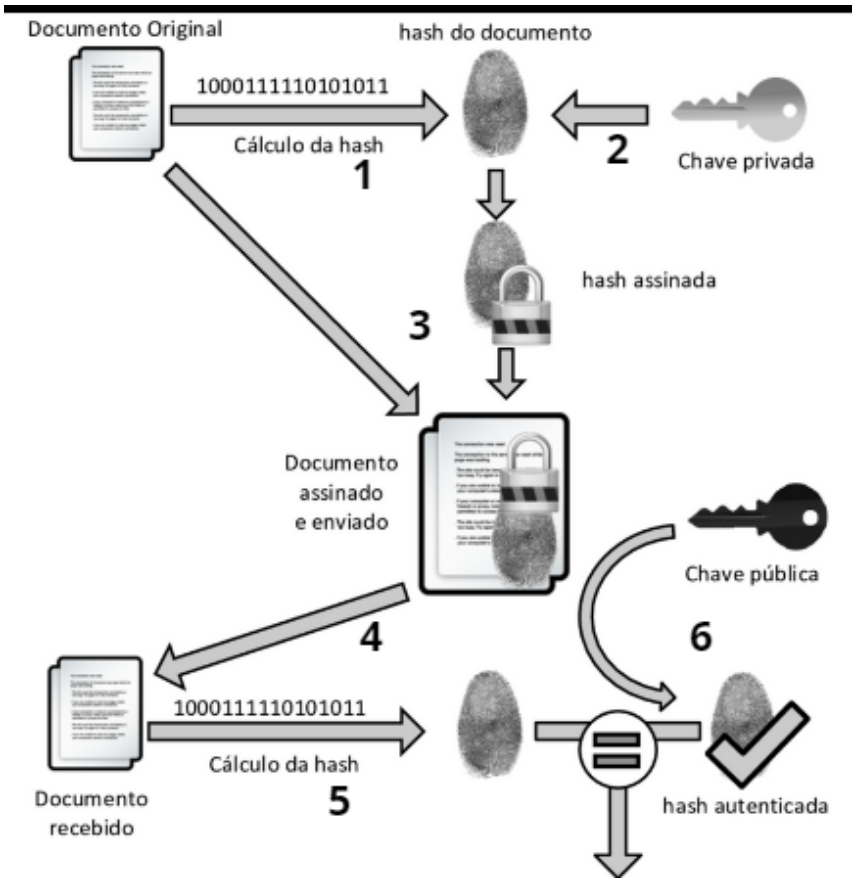
A Figura 3 de um lado apresenta exemplos de inputs de variados que através de uma função de hash criptográfico são transformados em saídas de um tamanho único, chamados de hash.

2.3.3 Assinaturas Digitais

Os hashes, isoladamente, não garantem a autenticidade nem a autoria dos dados, pois qualquer pessoa pode calcular o hash de uma mensagem sem comprovar sua origem. Além disso, hashes não oferecem mecanismos para verificar quem gerou ou modificou o conteúdo. Por isso, é necessário complementar as funções hash com métodos que assegurem a identidade do remetente e a integridade da informação, como as assinaturas digitais.

Para validar a autenticidade e a integridade de um documento digital ou de uma mensagem utilizamos Assinaturas Digitais. Utilizando a criptografia assimétrica o remetente utiliza sua chave privada para assinar digitalmente um documento, e o receptor utiliza a chave pública correspondente para verificar a assinatura (RIVEST; SHAMIR; ADLEMAN, 1978).

Figura 4 – Exemplo de Assinatura Digital



Fonte: (RODRIGUES, 2013)

Como podemos ver na Figura 4, o remetente utiliza sua chave privada para assinar a hash de um documento, gerando uma assinatura digital. O receptor, por sua vez, utiliza a chave pública correspondente para verificar a autenticidade da assinatura e a integridade do documento.

3 Identidade Digital

3.1 Síntese

Neste capítulo, abordamos a definição e os diferentes tipos de entidades que compõem uma identidade digital. Mostrando suas classificações e atributos, que permitem que essas entidades participem de interações de forma legítima e segura.

Exploramos ainda os principais tipos de sistemas de identidade digital: Centralizado, Federado e Descentralizado. Destacando o sistema descentralizado, impulsionado por tecnologias como o *blockchain*, devolvendo o controle das identidades aos próprios usuários, eliminando a necessidade de intermediários centralizados.

Focamos também nas Identidades Digitais Auto-soberanas (**SSIs**), que colocam o usuário no centro do controle de sua própria identidade. Apresentamos os 10 princípios fundamentais que garantem a autonomia, privacidade e segurança dos usuários nesse modelo, assegurando que os dados pessoais sejam geridos de forma independente e transparente.

Este capítulo estabelece as bases conceituais para a compreensão das Identidades Digitais Auto-soberanas, preparando para a exploração das tecnologias que suportam essas identidades no Capítulo 4 e nos capítulos seguintes.

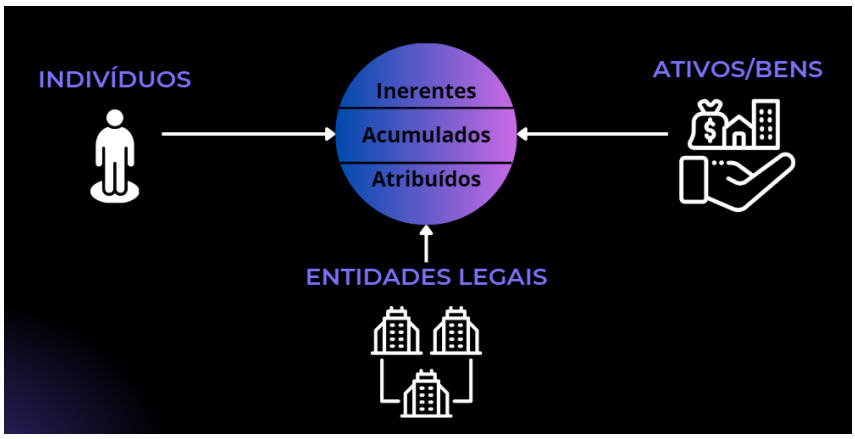
3.2 Definição e tipos de entidades

Identidade, seja física ou digital, é o conjunto de informações ou atributos individuais de uma entidade que são usados para determinar as interações nas quais a mesma pode participar legitimamente (MIT-IDE, 2021). Podemos dividi-la em três tipos de entidades, **Indivíduos**, **Entidades legais** e **Ativos/Bens**. Os atributos individuais dessas entidades também podem ser divididos em três categorias:

- **Inerentes**: Atributos que são intrínsecos a uma entidade e não são definidos por relações com entidades externas.
- **Acumulados**: Atributos que são reunidos ou desenvolvidos ao longo do tempo.
- **Atribuídos**: Atributos que são acrescentados à entidade, mas não estão relacionados à sua natureza intrínseca.

Esses atributos permitem que as entidades participem de interações, provando que possuem os atributos específicos necessários para aquela interação em particular (World Economic Forum, 2016).

Figura 5 – Tipos de entidades e seus atributos



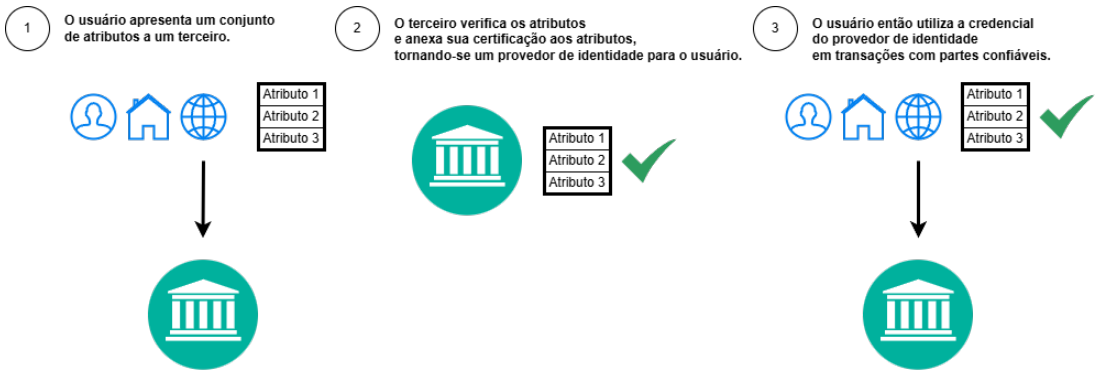
Fonte: Autoral

A Figura 5 ilustra os três tipos de entidades de uma identidade, e também as categorias dos atributos individuais que cada uma possue.

3.3 Sistemas de Identidade

Sabendo disso, podemos começar a pensar em uma estrutura para um sistema de identidades. Esse sistema tem o propósito de permitir que partes sem uma relação pré-estabelecida realizem transações com confiança. Os atributos dos usuários são atestados por terceiros confiáveis; Esses terceiros emitem credenciais que vinculam essa atestação aos atributos específicos, usando algum método de credencial para a entidade que a está representando. E para realizar transações com outras entidades que exijam alguma prova ou conhecimento de seus atributos os usuários podem utilizar carteiras de credenciais.

Figura 6 – Exemplo de Estrutura dos sistemas de identidade



Fonte: Autoral

A Figura 6 exemplifica um sistema de identidade onde no passo 1 O usuário apresenta um conjunto de atributos a um terceiro; no passo 2 esse terceiro verifica e certifica esses atributos assim se tornando o provedor de identidade, por fim no passo 3 o usuário utiliza essa credencial do provedor em transações com partes confiáveis.

A principal diferença entre um sistema tradicional físico para um sistema digital está apenas na forma com a qual os atributos são armazenados e as transações são realizadas, sendo tudo operado em ambiente digital, o que elimina a dependência de documentos físicos e processos manuais. Os principais tipos de sistemas de identidade atualmente são o **Centralizado**, **Federado** e o **Descentralizado**.

O **Centralizado** é onde uma única organização emite, controla e valida as identidades, como um banco, uma empresa ou o governo. Nesse modelo as informações armazenadas em servidores centralizados, facilitando o gerenciamento, mas criando um *Single Point of Failure*, tornando as informações mais vulneráveis à ataques e à perda de controle pelos usuários. Casos como o da violação de dados da Equifax em 2017 ilustram essas vulnerabilidades de maneira alarmante (Federal Trade Commission, 2024). Esses eventos recorrentes destacam a falta de controle e gestão que os usuários experimentam com suas identidades digitais, uma vez que não possuem autonomia sobre seus próprios dados e dependem exclusivamente das entidades centralizadoras para garantir a segurança de suas informações.

Por outro lado, o sistema **Federado** introduz uma rede de confiança entre várias entidades. Em vez de uma única autoridade central, diferentes provedores de identidade em uma federação compartilham as informações dos usuários de maneira segura e confiável. Com isso a mesma credencial pode ser utilizada em várias plataformas sem a necessidade de gerenciar múltiplas identidades separadas (CHADWICK, 2009).

Por fim, o sistema **Descentralizado** é uma abordagem que tem ganhado relevância com a adoção de tecnologias como o *blockchain*. Utilizando algoritmos criptográficos e registros distribuídos seguros, esses sistemas permitem a criação de provas matemáticas sobre a veracidade de atributos de identidade. Em um sistema descentralizado, o indivíduo armazena seus atributos de identidade e gerencia o ciclo de vida dos seus dados, escolhe quais informações compartilhar, com quem e para qual finalidade (Omar Dib, Khalifa Toumi, 2020).

Além disso, as Credenciais Verificáveis, que serão apresentadas em uma próxima seção, desempenham um papel central, permitindo que as informações sejam facilmente transferidas e verificadas de forma criptográfica. Essa abordagem melhora a experiência digital do usuário, promovendo maior controle e segurança para os indivíduos, mantendo a necessidade de terceiros confiáveis para emitir dados autenticados, como os condomínios (Omar Dib, Khalifa Toumi, 2020).

3.4 Identidades Digital Autos-soberanas

Focando na Identidade Digital Descentralizada, temos as Identidades Digitais Autos-soberanas ou **SSIs**. Que são aquelas na qual o indivíduo representado por aquela identidade possuem o total controle sobre ela sem a necessidade de intervenção de uma autoridade central (STOKKINK; POUWELSE, 2018).

Para atingir esse modelo de identidade auto-soberana temos 10 princípios com o objetivo de garantir que o controle do usuário esteja no centro do desenvolvimento de uma SSI (ALLEN, 2017). Esses princípios estão descritos e exemplificados na tabela a seguir.

Princípio	**Descrição**	**Exemplo**
Existência	Usuários devem existir de maneira independente.	A identidade não depende exclusivamente de uma plataforma online.
Controle	Usuários devem ter controle da identidade, sendo a autoridade máxima da sua identidade.	Escolher quais atributos compartilhar ao se registrar em um serviço.
Acesso	Total acesso aos dados de uma SSI ao usuário que detém o controle.	Visualizar todas as credenciais armazenadas em sua carteira digital.
Transparência	Os sistemas e algoritmos utilizados para administrar e operar uma rede de identidades devem ser abertos, tanto no modo como funcionam quanto na forma como são gerenciados e atualizados.	O código-fonte de uma plataforma SSI é *open-source*.
Persistência	Identidades devem existir para sempre, ou pelo menos por quanto tempo o detentor dela queira.	Mesmo que um provedor de serviço deixe de existir, a identidade do usuário permanece válida.
Portabilidade	Informações e serviços sobre a identidade precisam ser portáveis.	Migrar suas credenciais de uma carteira digital para outra.
Interoperabilidade	As identidades devem ser o mais amplamente utilizáveis possível.	Usar a mesma credencial em diversos serviços diferentes.
Consentimento	O compartilhamento de dados deve ocorrer apenas com o consentimento do usuário.	Autorizar ou negar o acesso a um atributo específico.
Minimização	A divulgação de informações deve ser minimizada, ou seja, envolver a menor quantidade de dados necessários para realizar a tarefa.	Provar que é maior de idade sem revelar a data de nascimento exata.
Proteção	Os direitos dos usuários devem ser protegidos.	Garantias legais contra o uso indevido de dados.

Tabela 1 – Princípios de sistemas de identidade descentralizada e seus exemplos

Esses princípios asseguram que o usuário mantenha o controle sobre suas informações pessoais, promovendo um ambiente digital mais seguro e confiável.

Existem tecnologias que suportam a descentralização e a segurança das identidades digitais, com isso garantimos que muitos desses princípios já estejam intrinsecamente cumpridos. Nas próximas seções, serão exploradas as abordagens tecnológicas que permitem a realização das SSIs, destacando como elas contribuem para atender aos princípios estabelecidos.

4 Tecnologias para Identidades Digitais Auto-soberanas

4.1 Síntese

Este capítulo trouxe uma base sólida e um pouco mais profunda sobre as tecnologias essenciais para a implementação eficaz das SSIs, preparando para a exploração das plataformas existentes que utilizam essas tecnologias no capítulo Capítulo 5.

Iniciamos com uma análise detalhada do **Blockchain**, destacando sua capacidade de fornecer uma infraestrutura descentralizada, segura e transparente para o armazenamento e gerenciamento de identidades digitais. Enfatizamos como os blockchains permissionados são particularmente adequados para ambientes empresariais e institucionais devido ao seu controle de acesso e maior eficiência.

Avançamos para os **Identificadores Descentralizados** (**DIDs**), que são fundamentais para a criação de identidades auto-soberanas. Abordamos a estrutura e o formato dos DIDs, bem como a flexibilidade que eles oferecem no controle e na gestão das identidades pelos próprios usuários.

Em seguida, discutimos as **Credenciais Verificáveis** (**VCs**), que complementam os DIDs ao permitir que informações sobre uma entidade sejam emitidas, armazenadas e verificadas de maneira segura e confiável.

Por fim, introduzimos as **Provas de Conhecimento Zero** (**ZKPs**), técnicas avançadas que aprimoram ainda mais a privacidade e a segurança das VCs. Discutimos diferentes tipos de ZKPs e apresentamos aplicações práticas, como as *Anoncreds*, que exemplificam como provar atributos específicos sem comprometer a privacidade do usuário.

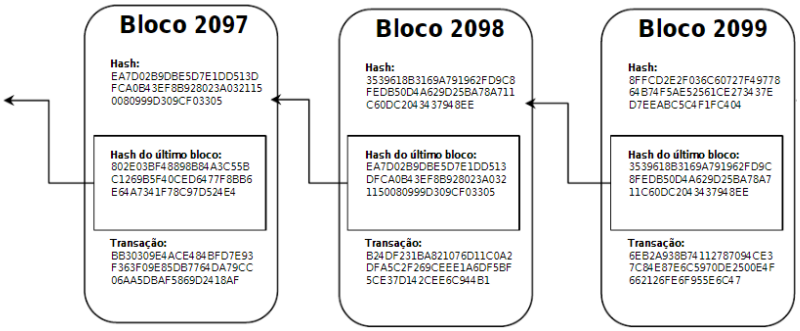
4.2 Blockchain

O **Blockchain** é uma tecnologia de registro distribuído que revolucionou a maneira como transações e informações são armazenadas e validadas (NAKAMOTO, 2008). Originalmente concebido como infraestrutura ao Bitcoin, o blockchain suporta aplicações além das criptomoedas, incluindo contratos inteligentes, cadeias de suprimentos e, crucialmente, sistemas de identidade digital (ZYSKIND; NATHAN; PENTLAND, 2015).

Ele eliminação da necessidade de uma autoridade central. Isso é alcançado através de mecanismos de consenso, entre os participantes da rede para validar e registrar transações. Cada bloco de transações é criptograficamente ligado ao bloco anterior, for-

mando uma cadeia inquebrável e resistente a alterações, o que assegura a **integridade** e a **imutabilidade** dos dados armazenados (ZYSKIND; NATHAN; PENTLAND, 2015).

Figura 7 – Exemplo de Blockchain



Fonte: (UFRJ, 2019)

A Figura 7 ilustra um exemplo de blockchain, onde cada bloco contém o seu hash, o hash do bloco anterior e as transações daquele bloco.

No contexto das Identidades Digitais Auto-soberanas (**SSIs**), o blockchain desempenha um papel fundamental ao fornecer uma infraestrutura confiável para o armazenamento e gerenciamento de identidades digitais.

4.2.1 Blockchain Permissionada

Blockchains permissionada são blockchains onde apenas um certo grupo de indivíduos podem acessa-la, aqueles que não fazem parte desse grupo permissionado não tem a capacidade de visualizar o conteúdo dos blocos e nem saber quem são os usuários daquela blockchain, permitindo que os participantes dessa blockchain mantenha uma certa anonimidade ao mundo de fora (GURURAJ, 2020).

Existem duas categorias principais de blockchains permissionadas as completamente privadas e as baseadas em consórcio também chamadas de blockchains federadas. Blockchains baseadas em consórcio são geridas por proprietários confiáveis, como governos e bancos, o direito de ler de um Blockchain pode ser público ou restringido a um certo grupo de participantes, por isso são frequentemente vistas como parcialmente descentralizadas. Já as blockchains totalmente privadas têm a permissão para escrever centralizada e gerida por uma única organização, embora a permissão para leitura ainda possa ser pública (MARTINOVIC LUCAS KELLO, 2017).

Agora vamos supor que os membros de uma blockchain possuam dados privados que desejam manter confidenciais, até mesmo para usuários que possuem permissão de ler dentro daquela blockchain. Para isso, esses dados devem ser armazenados em um dispositivo no qual o sujeito tenha controle total dos dados sem precisar que eles fiquem armazenados em um servidor central e nem na blockchain, e sempre que ele quiser compartilhar esses

dados poderá usar sua chave privada para criar assinaturas digitais. Essas assinaturas tem a capacidade de consentir em compartilhar aqueles dados e provar que essas informações são de sua autoria (UNIVERSITY, 2022).

Uma das maneiras de provar a autenticidade e autoria de um dado ou informação de uma maneira que mantenha a privacidade das informações é através da prova de zero conhecimento, ou zero knowledge proof (ZKP) (YANG; LI, 2020). Em uma próxima seção discutiremos como essas provas validam os dados de um sujeito sem precisar revelar as informações a quem interessa essa verificação.

4.3 Identificadores Descentralizados

Identificadores descentralizados (DIDs) são um tipo de identificador digital capaz de ser verificado, podendo provar controle sobre o identificador (SPORNY et al., 2022). DIDs também permitem que o sujeito ao qual o identificador se refere possa ter controle sobre o próprio ID, tornando-se assim uma Identidade auto-soberana (SSI) (Okta, 2024). Esse sujeito pode ser uma pessoa, um grupo de pessoas, uma organização ou algo qualquer.

O controller de uma DID não necessariamente precisa ser o sujeito, ou seja, o poder de alterar aquela DID pode ser delegado a um outro sujeito como, por exemplo, um pai que seria responsável por seu filho caso ele ainda não tenha a capacidade de gerenciar a chave pública da sua própria DID (KORTESNIEMI et al., 2019). Essa flexibilidade no controle é uma característica fundamental do formato da DID.

4.3.1 Formato DID

Uma DID é um identificador uniforme de recursos (URI), URI é uma sequência compacta de caracteres descrevendo o identificador de um recurso abstrato ou físico (BERNERS-LEE; FIELDING; MASINTER, 2005). Um DID é composto de três strings: o prefixo , "did"chamado de esquema, que precede o identificador do método usado e, por fim, o identificador específico de tal método. Métodos DID definem como os implementadores realizam as operações de criação, resolução, atualização e remoção de um DID (SPORNY et al., 2022).

Esta string método presente no DID funciona como se fosse um domínio em um site, como "example.com", indicando a origem e o contexto do identificador. Novos Métodos DID são definidos em suas próprias especificações para permitir a interoperabilidade entre diferentes implementações do mesmo método. A terceira parte de um DID é a String Específica do Método DID que por sua vez é um identificador público específico, composto por uma sequência de letras e números, que identifica de maneira única uma entidade ou objeto dentro do Método DID (ANDERSON, 2023).

Figura 8 – Exemplo simples de did

Fonte:(SPORNY et al., 2022)

A Figura 8 demonstra um exemplo simples de um Identificador Descentralizado (DID). O DID é composto por três componentes principais: o esquema ("did:"), o método DID ("example:"), e o identificador específico do método ("123456789abcdefghij"). O esquema "did:"indica que o identificador é um DID, o método DID especifica a abordagem ou o sistema utilizado para a criação do DID, e o identificador específico do método é a sequência única de caracteres que identifica o recurso de forma exclusiva dentro do método especificado. Esta estrutura permite a criação de identificadores únicos e verificáveis que são independentes de autoridades centralizadas, como mostrado em (SPORNY et al., 2022).

4.3.2 URL DID

Além disso, uma DID pode ter sua sintaxe estendida para uma URL DID, incorporando componentes adicionais de uma URI como o path (introduzido pelo caractere "/"), query (sendo "?" seu respectivo caractere) e fragmento (tendo "#" como o seu caractere). Uma URL DID é desreferenciada através de uma função para um documento DID ou recurso externo de acordo com os componentes encontrados no DID (SPORNY et al., 2022).

O componente path de uma URI contém dados que, juntamente com os encontrados no componente query, servem para identificar um recurso dentro do escopo do esquema e autoridade nomeadora da URI. O query é também frequentemente usado para transportar informações de identificação na forma de pares "chave=valor". Já o componente fragment é usado como uma forma que independe de método para referenciar um recurso externo ou uma propriedade encontrada no documento DID com por exemplo um método de verficação único (BERNERS-LEE; FIELDING; MASINTER, 2005).

O exemplo a seguir é um exemplo de DID contendo os componentes path, query e fragment, /path/to/resource refere-se a um recurso específico relacionado ao DID, key1=value1 são parâmetros de consulta que podem ser usados para passar informações adicionais ao resolver o DID, section refere-se a uma parte específica dentro do recurso

identificado pelo path.

Figura 9 – Componentes DID

```
did:example:123456789abcdefghi/path/to/resource?key1=value1#section
```

Fonte:(SPORNY et al., 2022)

A Figura 9 exibe um exemplo dos componentes de um DID URL. O DID URL é composto por vários elementos: o caminho ("path/to/resource"), que direciona para um recurso específico de forma hierárquica; a consulta ("?key1=value1"), que adiciona parâmetros de consulta para passagem de dados adicionais; e o fragmento ("#section"), que indica uma referência a uma seção específica dentro do recurso. Esta estrutura permite a localização e a referência de recursos específicos e subseções, de forma única e verificável, dentro de um sistema descentralizado.

4.3.3 Documento DID

Um documento DID é o resultado da resolução de um DID que é resolvido através de um DID resolver, um DID resolver é um hardware e/ou software que recebe como input um DID e libera como output um documento DID.(SPORNY et al., 2022).

Figura 10 – Exemplo simples de um documento did.

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/ed25519-2020/v1"
  ]
  "id": "did:example:123456789abcdefghi",
  "authentication": [{
    // used to authenticate as did:...fghi
    "id": "did:example:123456789abcdefghi#keys-1",
    "type": "Ed25519VerificationKey2020",
    "controller": "did:example:123456789abcdefghi",
    "publicKeyMultibase": "zH3C2AVvLMv6gmMNam3uVAjZpfkcJCwDwnZn6z3wXmqPV"
  }]
}
```

Fonte: (SPORNY et al., 2022)

A Figura 10 é uma representação de um documento DID contendo um id, propriedades de autenticação e a propriedade @context nos entrega a forma como o documento deve ser processado de acordo com as regras do JSON-LD. Caso isso não seja possível ou o processamento falhe, o documento deve ser rejeitado como um documento did,, se o @context estiver presente, passar pelo processamento do JSON-LD e contiver o contexto

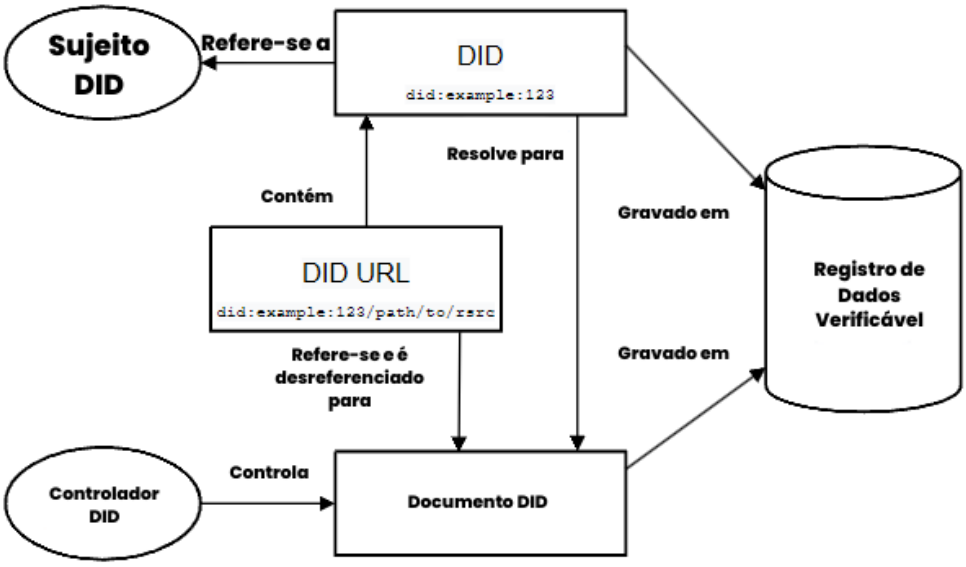
"https://www.w3.org/ns/did/v1", ele poderá ser processado como um documento DID de acordo com a especificação encontrada em (SPORNY et al., 2022).

Documentos DID contém informações sobre um determinado DID como o seu id que é a string mostrada na Figura 8 o id é uma propriedade obrigatória em um documento pois ela é quem identifica o DID subject daquele DID (SPORNY et al., 2022). Documentos podem incluir um controller que é uma string ou um conjunto de strings que referenciam o id de quem tem controle sobre a DID desse documento podendo ser o próprio sujeito daquela DID como dito anteriormente.

4.3.4 Arquitetura DID

Os conceitos anteriores como DID subject, DID, DID controller, se relacionam de acordo com a imagem a seguir.

Figura 11 – Arquitetura DID.



Fonte: Adaptado de (SPORNY et al., 2022)

A Figura 11 mostra um diagrama relacionado ao sistema de Identificadores Descentralizados (DIDs), que são uma parte fundamental da infraestrutura de identidades descentralizadas onde nela encontramos o DID Subject que representa o sujeito ou entidade para a qual o DID é criado, DID: O Identificador Descentralizado em si que se refere ao DID Subject e é resolvido para o DID document.

DID URL contém o DID e caminhos adicionais que referenciam e "desreferenciam"(dereferenciam) o recurso ou o documento DID. DID Document e DID são registrado em um Registro de Dados Verificáveis, que garante que os dados associados ao DID sejam verificáveis de forma confiável e pública.

4.4 Credenciais Verificáveis

As *Credenciais Verificáveis* são um componente essencial no ecossistema de identidades descentralizadas, complementando os *Identificadores Descentralizados* (DIDs). As Credenciais Verificáveis permitem que informações sobre uma entidade sejam emitidas, armazenadas e verificadas de maneira segura e confiável, sem a necessidade de intermediários centralizados (SPORNY; LONGLEY; CHADWICK, 2022).

4.4.1 Definição e Estrutura

Uma *Credencial Verificável* é uma declaração sobre um sujeito (*credentialSubject*) emitida por uma autoridade confiável (*issuer*) e pode ser verificada por terceiros (*verifier*) sem a necessidade de comunicação direta com o emissor. As Credenciais Verificáveis são baseadas na recomendação W3C, garantindo interoperabilidade e consistência entre diferentes implementações (SPORNY; LONGLEY; CHADWICK, 2022).

A estrutura básica de uma *Credencial Verificável* inclui:

Elemento	**Descrição**
Contexto (@context)	Define o contexto semântico da credencial, permitindo que diferentes sistemas interpretem os dados de forma consistente.
ID da Credencial (id)	Identificador único da credencial.
Tipo (type)	Especifica os tipos de credencial, como "VerifiableCredential".
Emissor (issuer)	Referência ao DID do emissor da credencial.
Sujeito (credentialSubject)	Contém as declarações sobre a entidade à qual a credencial se refere.
Propriedades de Emissão (issuanceDate)	Data em que a credencial foi emitida.
Assinatura Digital (proof)	Garante a integridade e a autenticidade da credencial.

Tabela 2 – Elementos de uma Credencial Verificável e suas descrições

4.4.2 Processo de Emissão, Apresentação e Verificação

O ciclo de vida de uma *Credencial Verificável* envolve três etapas principais:

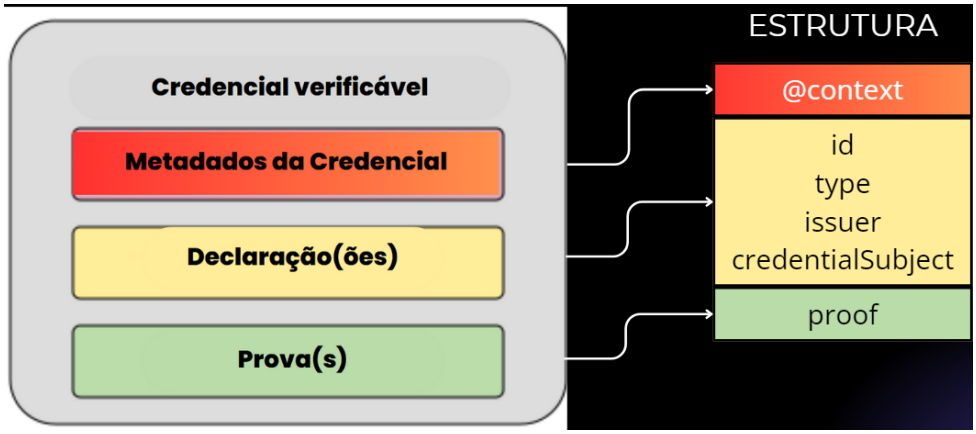
1. **Emissão**: O emissor cria a credencial contendo as declarações sobre o sujeito e assina digitalmente a credencial utilizando sua chave privada associada ao seu DID.

2. **Apresentação**: O sujeito armazena a credencial em sua carteira digital e pode apresentá-la a um verificador conforme necessário.

3. **Verificação**: O verificador utiliza a chave pública do emissor (obtida através do DID do emissor) para verificar a assinatura digital da credencial, assegurando sua autenticidade e integridade.

Isso garante que as informações apresentadas são confiáveis e que a identidade do emissor pode ser verificada sem depender de uma autoridade central (SPORNY; LONGLEY; CHADWICK, 2022).

Figura 12 – Exemplo de Estrutura de uma Credencial Verificável



Fonte: Adaptado de (SPORNY; LONGLEY; CHADWICK, 2022)

A Figura 12 ilustra um exemplo de uma *Credencial Verificável*, destacando seus principais componentes, Credential Metadata, Claim(s) e Proof(s).

4.4.3 Vantagens das Credenciais Verificáveis

As *Credenciais Verificáveis* oferecem diversas vantagens no contexto de identidades digitais descentralizadas:

Características	Descrição
Autonomia do Usuário	Permitem que os usuários mantenham controle total sobre suas credenciais, armazenando-as em carteiras digitais pessoais (ALLEN, 2017).
Privacidade e Seleção de Dados	Os sujeitos podem compartilhar apenas as informações necessárias, preservando a privacidade e reduzindo a exposição de dados sensíveis.
Interoperabilidade	Baseadas em padrões abertos, as Credenciais Verificáveis são compatíveis com diferentes sistemas e plataformas, facilitando a adoção ampla (BUTTERWORTH; CURREN, 2022).
Segurança e Integridade	A utilização de assinaturas digitais e DIDs garante que as credenciais não sejam adulteradas e que a origem das informações seja verificável.

Tabela 3 – Características das Credenciais Verificáveis

4.4.4 Aplicações das Credenciais Verificáveis

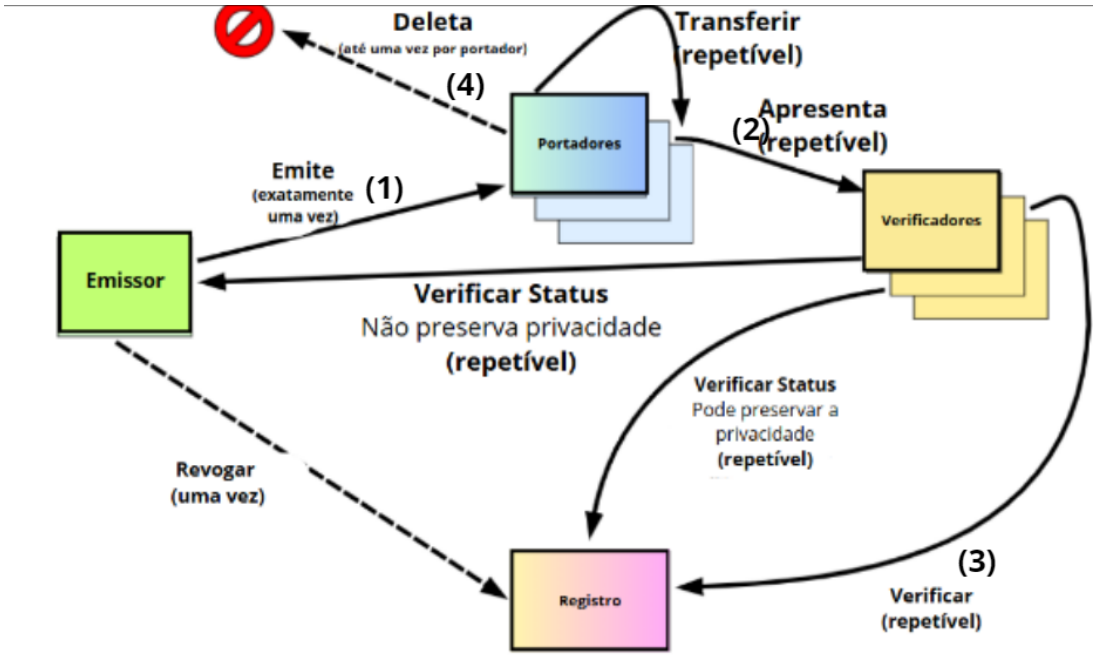
As *Credenciais Verificáveis* têm uma ampla gama de aplicações, incluindo:

Domínio	Descrição
Identidade Pessoal	Prova de identidade para serviços online, abertura de contas bancárias, entre outros.
Educação	Diplomas e certificados acadêmicos verificáveis.
Saúde	Registros médicos e certificados de vacinação.
Finanças	Prova de renda e histórico de crédito.
Condomínios	Gestão de acesso e permissões em condomínios residenciais e comerciais, facilitando a autenticação de moradores e visitantes, controle de acesso a áreas restritas, e verificação de credenciais para prestadores de serviços.

Tabela 4 – Domínios de Aplicação de Credenciais Verificáveis

Essas aplicações demonstram a versatilidade das Credenciais Verificáveis em diferentes setores, promovendo uma gestão de identidade mais segura e eficiente (HERBKE; SAPKOTA; LAMICHHANE, 2024).

Figura 13 – Ciclo de Vida de uma Credencial Verificável



Fonte: Adaptado de (SPORNY; LONGLEY; CHADWICK, 2022)

A Figura 13 ilustra o ciclo de vida de uma *Credencial Verificável*, destacando as etapas de emissão, apresentação e verificação, e a interação entre o emissor, o sujeito e o verificador.

4.5 Provas de Conhecimento Zero

Para aprimorar ainda mais a privacidade e a segurança das *Credenciais Verificáveis*, técnicas avançadas como *Zero-Knowledge Proofs* (ZKP) são integradas. As ZKPs permitem a verificação de informações sem revelar os dados subjacentes, proporcionando um nível adicional de proteção para a identidade auto-soberana.

4.5.1 Definição e Princípios Fundamentais

Provas de zero conhecimento, ou Zero-Knowledge Proofs (ZKP), são protocolos criptográficos que permitem a uma parte (*prover*) demonstrar a outra parte (*verifier*) que uma determinada afirmação é verdadeira, sem revelar qualquer informação adicional além da veracidade da afirmação (CHAUM, 1985). Em outras palavras, uma ZKP permite que um usuário prove que conhece um valor específico sem divulgar o próprio valor, garantindo assim a privacidade e a segurança dos dados envolvidos.

4.5.2 Provas de Conhecimento Zero em Credenciais Verificáveis

Segundo a Recomendação W3C, as ZKPs oferecem diversos atributos que são compatíveis com *Credenciais Verificáveis* (VCs), incluindo:

- **Divulgação Seletiva**: Permite que o detentor da credencial forneça apenas as informações necessárias ao verificador, sem expor outros dados presentes na credencial (SPORNY; LONGLEY; CHADWICK, 2022). Por exemplo, um indivíduo pode provar que tem mais de 25 anos sem revelar sua data de nascimento completa.

- **Divulgação não vinculável**: Utilizando assinaturas cegas, as ZKPs possibilitam que a divulgação de informações não possa ser correlacionada entre múltiplas apresentações a diferentes verificadores, aumentando a privacidade do detentor (SPORNY; LONGLEY; CHADWICK, 2022).

- **Identificação Não Correlacionável**: Permite que o detentor prove a emissão de uma credencial sem compartilhar um identificador que possa ser correlacionado em diferentes interações, reduzindo a quantidade de dados compartilhados e aumentando a segurança (SPORNY; LONGLEY; CHADWICK, 2022).

Essas capacidades são fundamentais para garantir que as *Credenciais Verificáveis* sejam utilizadas de forma segura e privada, alinhando-se com os princípios da Identidade auto-soberana.

4.5.3 Tipos de Provas de Conhecimento Zero

Existem diversos tipos de ZKPs, cada um adequado para diferentes contextos e aplicações. Entre os mais conhecidos estão apresentados na Tabela 5.

Tabela 5 – Tipos de Provas de Conhecimento Zero

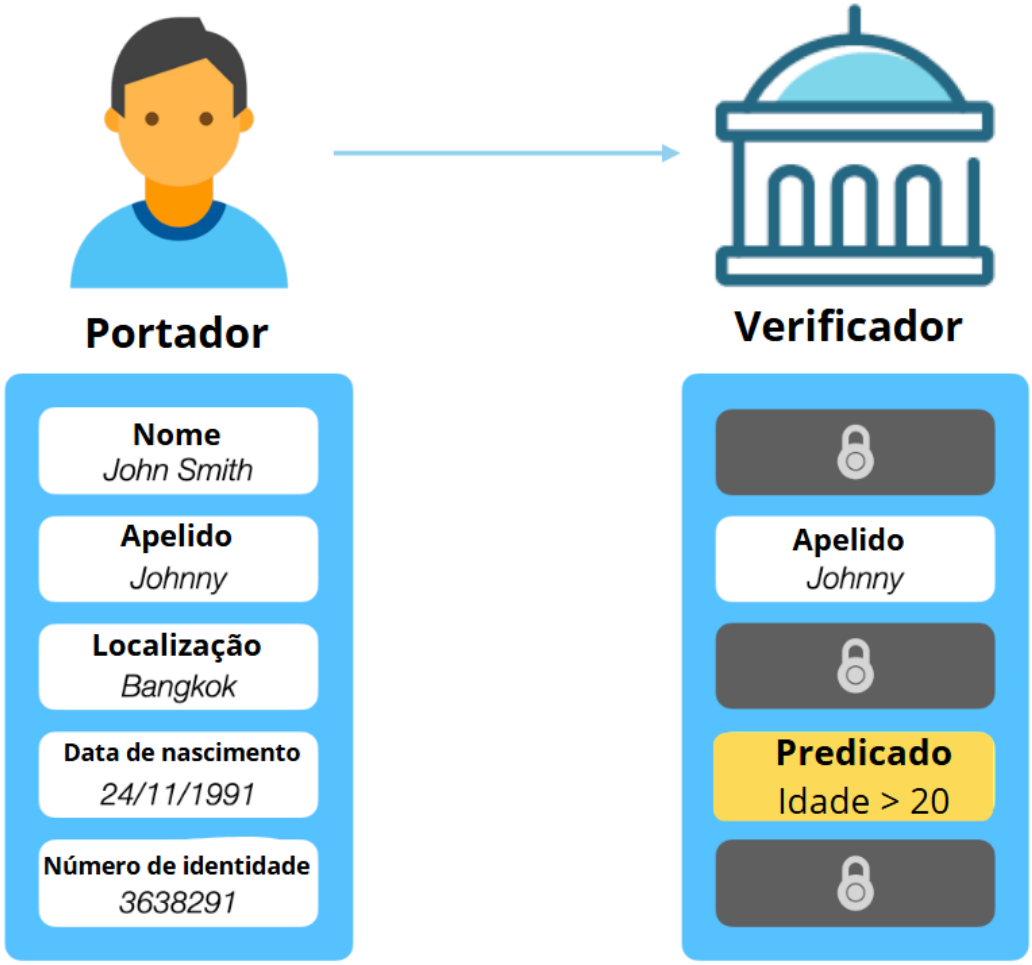
Tipo de ZKP	**Descrição**
Provas de Conhecimento Zero de um Segredo	Permite que uma parte prove que conhece um segredo específico sem revelar o segredo em si (ZHOU et al., 2024).
Provas de Conhecimento de uma Afirmação	Habilita uma parte a demonstrar que conhece uma solução para uma determinada afirmação ou equação, sem revelar como chegou a essa solução (ZHOU et al., 2024).
Provas de Conhecimento Zero de Identidade	Permite que uma parte prove sua identidade sem revelar informações adicionais sobre si mesma, aumentando a privacidade na autenticação (ZHOU et al., 2024).

4.5.4 Anoncreds: Uma Aplicação Prática das ZKPs

Uma das aplicações práticas das ZKPs é o conceito de *Anoncreds* (Anonymous Credentials), que é a capacidade de provar atributos sobre um indivíduo sem revelar sua identidade completa (BRANDS, 1995). As *Anoncreds* permitem que um usuário prove que possui determinadas qualidades ou características sem expor quem ele é, garantindo assim a privacidade e a segurança das informações.

Por exemplo, existem conteúdos e serviços online que tem restrição de idade para poderem ser acessados, um usuário pode utilizar uma *Anoncred* para provar que tem idade suficiente sem revelar sua data de nascimento completa. Isso é particularmente útil em cenários onde a verificação da autenticidade das informações é necessária sem comprometer a privacidade do indivíduo.

Figura 14 – Exemplo de Uso de *Anoncred*



Fonte: Adaptado de (KONGSUWAN; TOSIRISUK, 2020)

A Figura 14 ilustra um exemplo de uso de uma *Anoncred*. De um lado, está o *Holder* (portador das credenciais), representado como uma pessoa com atributos armazenados, como Nome, Apelido, Endereço, Data de Nascimento e Número de Identidade. Do outro lado, está o Verifier (verificador), representado por um ícone de instituição, que pode verificar informações específicas do Holder.

No exemplo, o Holder compartilha seletivamente apenas o atributo Apelido e uma afirmação derivada do atributo Data de Nascimento, expressa como um predicado: Idade > 20, sem revelar sua data completa de nascimento ou outras informações pessoais. Os outros atributos permanecem bloqueados, garantindo a privacidade.

Essa abordagem demonstra como as *Anoncreds* podem ser usadas para verificar características ou qualificações de um indivíduo sem expor sua identidade completa, proporcionando maior segurança e preservação de dados.

5 Plataformas de Identidade Auto-soberana

Por ser um modelo de identidade considerado novo e com muitos projetos ainda em sua fase inicial, o conceito de **Identidade Digital Auto-soberana** (**SSI**) vem ganhando visibilidade mundial, com algumas organizações fundadas com foco exatamente nessa área. Com um bom potencial de revolucionar a abordagem tradicional para a gestão de identidades, permitindo que indivíduos mantenham o controle completo sobre seus próprios dados pessoais, sem depender de autoridades centralizadas.

5.1 Síntese

Neste capítulo, foram exploradas as plataformas de SSI, abordando seu papel no gerenciamento descentralizado de identidades. Foram analisadas plataformas como Hyperledger Aries, Sovrin e Veramo, destacando suas características e aplicações práticas. Escolhemos o Hyperledger Aries/ACA-Py para a implementação por sua descentralização baseada em blockchain, em conformidade com as recomendações W3C para privacidade, facilidade de implementação e um ótimo suporte da comunidade. Esta plataforma foi reconhecida por sua modularidade e interoperabilidade, permitindo o desenvolvimento de soluções SSI flexíveis e seguras. Este capítulo estabelece as bases para a utilização do Hyperledger na prova de conceito apresentada no capítulo seguinte.

5.2 Hyperledger Foundation

O Hyperledger foundation é uma iniciativa de código aberto com o objetivo de promover o desenvolvimento de tecnologias blockchain que atendam uma ampla gama de casos de uso (FOUNDATION, 2024). Fundado pela Linux Foundation em dezembro de 2015 e apoiado por diversas organizações líderes em tecnologia (Linux Foundation, 2015), o Hyperledger reúne uma comunidade global de desenvolvedores, pesquisadores e entusiastas que colaboram para aprimorar e expandir o ecossistema de blockchains empresariais.

O Hyperledger Foundation, através da sua tecnologia Hyperledger Besu, está presente no desenvolvimento do real digital, sendo a blockchain escolhida para o desenvolvimento da moeda devido a capacidade de controle e privacidade, disse Haroldo Jayme Cruz, chefe do departamento de informática do Real Digital, em coletiva de imprensa realizada no dia *06/03/2023*.

5.2.1 Projetos Hyperledger

A Hyperledger foundation engloba diversos projetos que oferecem ferramentas e frameworks para o desenvolvimento de aplicações blockchain. A seguir, são apresentados alguns dos principais projetos do ecossistema Hyperledger:

- **Hyperledger Indy**: É um software de *Distributed Ledger Technology* (DLT) que oferece ferramentas e componentes reutilizáveis para fornecer identidades digitais baseadas em blockchains, garantindo interoperabilidade entre diferentes domínios administrativos e aplicações (FOUNDATION, 2024d). Pode ser utilizado de forma independente ou integrado a outras blockchains, promovendo a descentralização da identidade.

- **Hyperledger Aries**: Fornece um conjunto de ferramentas focadas na criação, transmissão e armazenamento de credenciais digitais verificáveis. Serve como infraestrutura para interações *peer-to-peer* baseadas em blockchain (FOUNDATION, 2024b).

- **Hyperledger Fabric**: Software de DLT que serve como base para o desenvolvimento de aplicações com uma arquitetura modular, permitindo que componentes como consenso e serviços de membros sejam plug-and-play, atendendo a uma ampla gama de casos de uso (FOUNDATION, 2024c).

- **Hyperledger Anoncreds**: Ferramenta que oferece recursos avançados de proteção de privacidade, incorporando ZKP para reforçar a privacidade sem comprometer as garantias centrais das credenciais verificáveis (FOUNDATION, 2024a).

5.3 Comparação com Outras Plataformas de SSI

É essencial comparar os projetos do Hyperledger foundation com outras plataformas para entender suas diferenças, vantagens e limitações, facilitando a escolha da mais adequada para diferentes casos de uso.

5.3.1 Sovrin

A **Sovrin Network** é uma rede global de identidade descentralizada baseada em *DLT*, projetada especificamente para SSIs (SOVRIN, 2018). Foi a primeira base para o começo do desenvolvimento do Hyperledger Indy, fornecendo uma infraestrutura confiável e segura para a gestão de identidades digitais, permitindo que indivíduos mantenham o controle total sobre seus dados pessoais. Por isso é mais interessante seguir o caminho dos projetos Hyperledger para desenvolver a prova de conceito, pois eles permitem um desenvolvimento mais flexivel sem depender de uma rede específica como a Sovrin.

5.3.2 uPort/Veramo

Desenvolvida pela **Consensys**, a **uPort** era uma plataforma de identidade descentralizada que permitia aos usuários criar e gerenciar suas identidades digitais de forma auto-soberana (NAIK; JENKINS, 2020). Em 2021 o uPort foi descontinuado para dar espaço para o novo projeto **Veramo**, que é um framework em JavaScript com os mesmos objetivos do uPort e para incentivar desenvolvedores a adicionar uma camada a mais de segurança em suas aplicações.

Escolhemos seguir o desenvolvimento com a plataforma do Hyperledger, mais especificamente o **Hyperledger Aries** utilizando o framework de desenvolvimento deles **ACA-Py**, que será explicado mais profundamente no próximo capítulo. As motivações para seguir com essa plataforma foram as seguintes:

- **Descentralização**: O Hyperledger Aries é fundamentado em tecnologia blockchain, utilizando o Hyperledger Indy para todas funcionalidades relacionadas à blockchain.
- **Privacidade**: A plataforma oferece recursos avançados de proteção de privacidade, todas seguindo as recomendações do W3C.
- **Facilidade de Implementação**: O Hyperledger Aries/ACA-Py é muito flexivel e com uma arquitetura modular. Além de estar acompanhado de uma boa documentação e APIs, o que facilita o desenvolvimento de uma solução SSI.
- **Comunidade e Suporte**: A disponibilidade de exemplos e demonstrações suportam muito o início, quando estamos aprendendo a utilizar a plataforma. Além disso, a comunidade Hyperledger é muito ativa e oferece suporte extenso para os desenvolvedores interessados, desde suporte nos Issues do GitHub até servidor no Discord com membros muito ativos.

6 Segurança de condomínios

A maneira que a liberação de acessos é feita atualmente em condomínios é um processo que pode não ser eficiente e seguro. Em boa parte dos condomínios, a liberação de acesso ainda é feita manualmente por porteiros, que muitas vezes não possuem um controle eficiente de quem entra e sai do condomínio. Até mesmo os condomínios que possuem tecnologias de controle de acesso mais modernas, como biometria e reconhecimento facial, ainda possuem falhas graves de segurança, como vazamento de dados pela empresa terceirizada responsável ou até mesmo falhas no sistema de segurança.

Em maio de 2024, um incidente em Jundiaí evidenciou a vulnerabilidade dos sistemas de segurança em condomínios. Centenas de moradores tiveram seus dados pessoais, como e-mail, CPF, RG, endereço, telefones e fotos, expostos na dark web e em grupos do Telegram. O ataque foi feito ao sistema de reconhecimento facial de uma empresa terceirizada que gerenciava vários condomínios na região. Embora a empresa afirme que nenhuma informação tenha sido roubada dos servidores, especialistas em segurança da informação afirmam que é possível que o sistema tenha sido invadido sem deixar rastros ou até mesmo que os dados tenham sido vazados por alguém com acesso ao sistema (G1, 2024).

Esse caso ressalta a importância de um sistema de controle de acesso seguro, que seja independente, transparente e confiável. E é por isso que o uso de SSI em condomínios pode trazer diversas vantagens e resolver muitos problemas que essa área enfrenta atualmente. Permitindo que os moradores tenham controle sobre suas próprias identidades e que a liberação de acesso seja feita de forma segura e eficiente.

Com uma aplicação de controle de acesso baseada em SSI, temos o condomínio como entidade emissora de credenciais e os moradores como titulares dessas credenciais. Dessa maneira, o morador detém o controle sobre quais informações ele deseja compartilhar sem comprometer sua privacidade e seu acesso ao condomínio. Um nível adicional de segurança passa a existir quando a criptografia e a descentralização do SSI passam a ser utilizadas.

7 Implementação da Prova de Conceito

7.1 Síntese

Neste capítulo, foi apresentada a implementação da prova de conceito de um sistema de identidade auto-soberana (SSI) aplicado ao controle de acesso em condomínios residenciais. Foram detalhadas as tecnologias utilizadas, como Python (com o framework ACA-Py), React, Docker, OpenAPI. Em seguida, detalhamos a blockchain da nossa aplicação Hyperledger Indy node, utilizada através de uma rede von-network como sendo o software de registro distribuído que nos permite executar as diversas operações necessárias para o funcionamento da aplicação.

Por fim, descrevemos a arquitetura da aplicação, enfatizando os fluxos de conexão entre agentes, emissão de credenciais e verificação, além de demonstrar o fluxo de trabalho das requisições na API REST da emissão e verificação de credenciais.

7.2 Tecnologias usadas

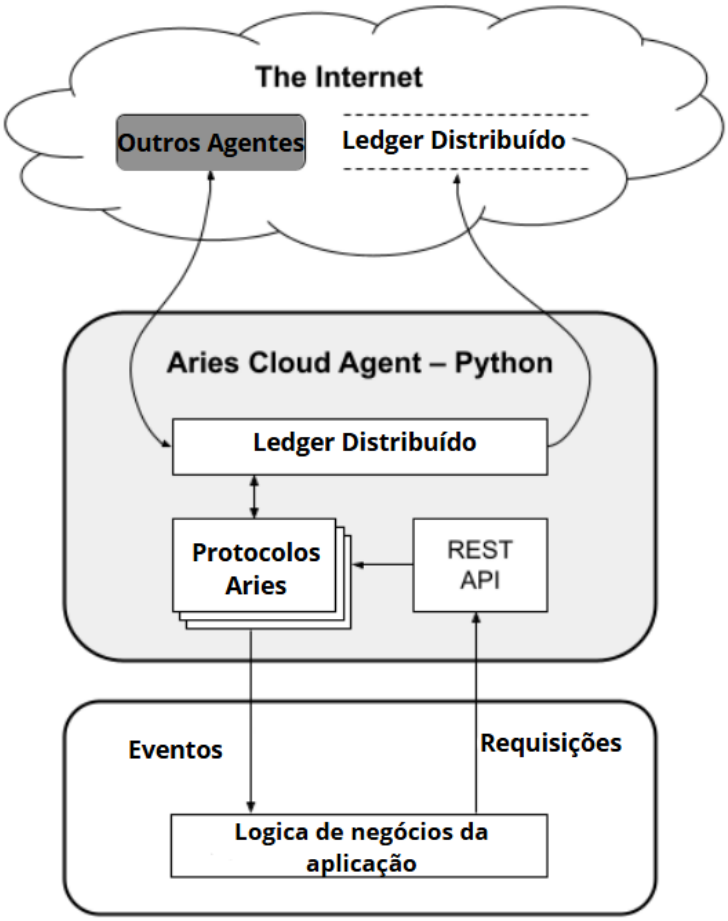
7.2.1 Python

Python é uma linguagem de programação de alto nível, de tipagem forte e dinâmica. Sua simplicidade e vasto ecossistema de bibliotecas a tornam ideal para soluções voltadas a blockchain e identidades descentralizadas (Python Software Foundation, 2023).

7.2.1.1 ACA-Py

ACA-Py é uma implementação de agente baseado na especificação do Hyperledger Aries, desenvolvida em Python. Ele permite a construção de aplicações compatíveis com credenciais verificáveis, DIDs e interações baseadas em protocolos DIDComm. Este agente fornece suporte completo para interações *peer-to-peer*, como troca de credenciais e verificações criptográficas, e é projetado para ser extensível e personalizável (Hyperledger Foundation, 2024).

Figura 15 – Arquitetura do Acapy



Fonte: Adaptado de (Hyperledger Foundation, 2024)

A Figura 15 ilustra a arquitetura do ACA-Py, dividida em três camadas principais. Na camada superior, estão os Agentes e a DLT, que representam os sistemas externos e a blockchain, respectivamente. A camada do meio é onde o ACA-Py se encontra, responsável pelas capacidades centrais e pelo gerenciamento de protocolos do Aries e da API REST para comunicação. Na camada inferior, o "Controller"integra a lógica de negócios da aplicação, processando eventos recebidos da camada intermediária e enviando requisições.

7.2.1.1.1 OpenAPI

OpenAPI, integrado ao ACA-Py, oferece uma interface de programação de aplicativos (API) padronizada e aberta para interação com serviços do agente (OpenAPI Initiative, 2024). Simplificando o desenvolvimento ao fornecer uma documentação interativa e suporte a diversos frameworks, facilitando a integração do ACA-Py em aplicações distribuídas (OpenWallet Foundation, 2024).

7.2.2 React

React é um *Framework* JavaScript de código aberto para a construção de interfaces de usuário. Desenvolvido pelo Facebook, o React é amplamente utilizado devido à sua abordagem baseada em peças individuais chamadas componentes (Meta Platforms, Inc., 2023). Na nossa solução de identidade descentralizada, o React foi escolhido para a criação do dashboard e interfaces de interação com agentes.

7.2.3 Docker

Docker é uma plataforma de conteinerização que nos permite empacotar e executar aplicações em ambientes isolados. Ele simplifica o gerenciamento de dependências e a implantação de soluções, sendo uma escolha popular em ecossistemas blockchain devido à necessidade de replicar ambientes consistentes (Docker Inc., 2023). Projetos como o Hyperledger Indy Node e o Von-Network utilizam contêineres Docker para configuração de redes locais e integração de ferramentas.

7.3 Tecnologia Blockchain

Para o desenvolvimento do nosso projeto, foi utilizado o software de registro distribuído Indy-node, através da rede von-network.

7.3.1 Indy-node

Indy Node é o responsável por gerenciar os nós da nossa rede Hyperledger Indy. Cada nó é um participante da rede que executa operações críticas, como validação de transações, consenso distribuído e manutenção do estado do livro-razão (Linux Foundation, 2024). Algumas características principais incluem:

- **Gerenciamento de identidades**: Suporta a criação, atualização e revogação de identidades descentralizadas (DIDs – *Decentralized Identifiers*) e seus atributos associados.
- **Operações em rede permissionada**: O Indy Node opera em um ambiente onde os participantes são conhecidos e confiáveis como em um condomínio residencial.

7.3.1.1 VON-network

Von-Network é uma implementação prática de rede Hyperledger Indy usada para fins de desenvolvimento e teste. Ela permite a criação rápida de uma rede local de nós baseados em Indy Node, fornecendo um ambiente ideal para validar funcionalidades antes da implantação em redes de produção (BCgov, 2017).

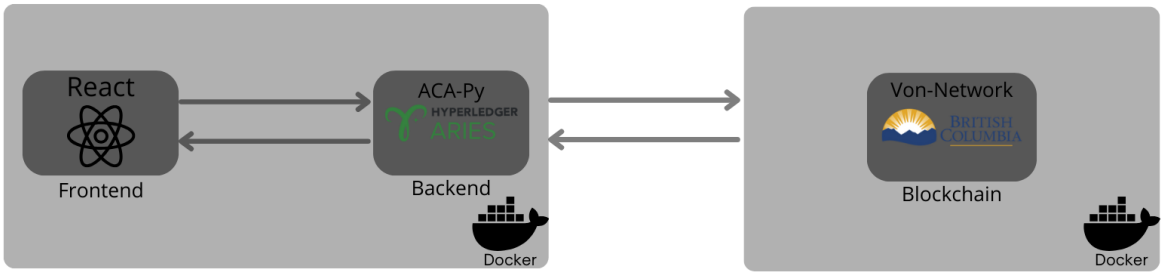
Principais vantagens do uso da von-network são: ambiente local de desenvolvimento facilitando a configuração de redes distribuídas em contêineres Docker; simulação de transações, podemos criar e testar transações de identidade, como registro de DIDs e operações de credenciais verificáveis, em um ambiente controlado. Flexibilidade para testes de integração: a Von-Network é compatível com outras ferramentas do ecossistema Hyperledger, como o Aries e o AnonCreds, permitindo testes *peer-to-peer* em soluções de DID. (BCgov, 2017)

7.4 Aplicação

7.4.1 Arquitetura

A arquitetura da aplicação foi dividida em três simples partes: Frontend (React), Backend (ACA-Py) e Blockchain (Von-Network). O frontend lida apenas com a interface do usuário e a comunicação com a API do backend. O backend é responsável por gerenciar os agentes e a comunicação com a blockchain. E a blockchain é onde o Schema, que é um modelo de dados para credenciais verificáveis, definições de credencial e transações, são registrados.

Figura 16 – Arquitetura da aplicação

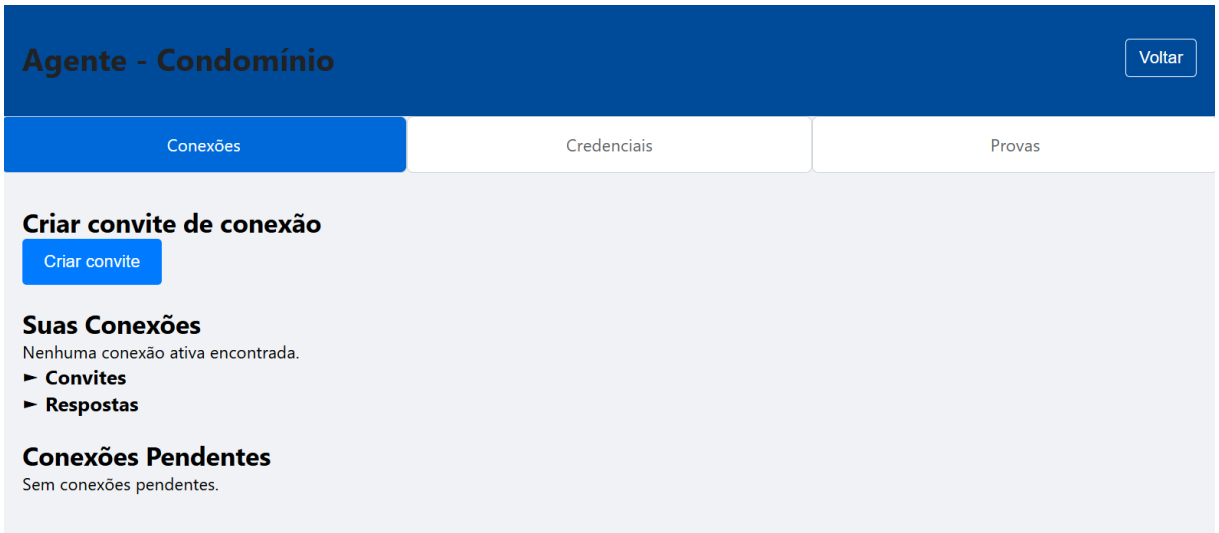


Fonte: Autoral

7.4.1.1 Frontend

O *frontend* da aplicação foi desenvolvido utilizando o React, projetando a interface para ser prática e intuitiva. A aplicação foi dividida em duas partes principais: o dashboard do condomínio e o dashboard do residente. O dashboard do condomínio é onde o administrador do condomínio pode fazer conexões, emitir credenciais para os residentes e fazer a verificação das mesmas para a liberação de acesso. Já o dashboard do residente é onde o morador aceita convites de conexão, solicita credenciais e apresenta provas de credenciais para acessar o condomínio.

Figura 17 – Dashboard do Condomínio



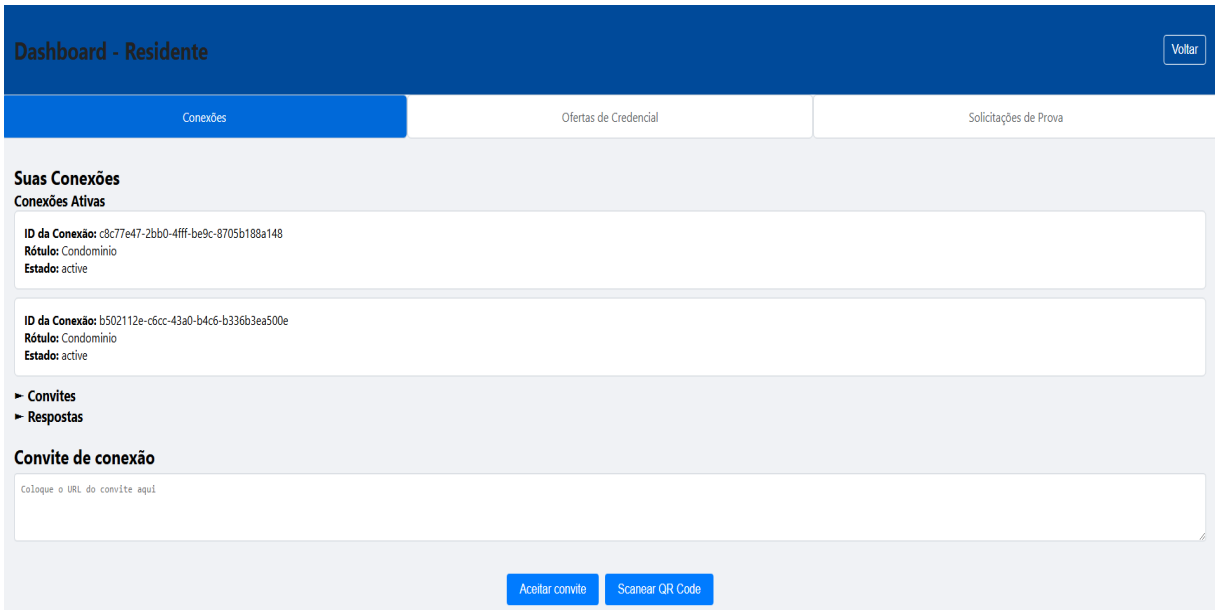
Fonte: Autoral

Figura 18 – Criação de Schema (Condomínio)

Fonte: Autoral

A Figura 17 mostra o dashboard do condomínio, onde o administrador pode gerenciar as credenciais dos residentes e fazer a verificação delas para liberação do acesso. Na Figura 18 é possível ver a tela de criação de um schema onde o condomínio pode definir quais atributos deseja que os residentes tenham em suas credenciais.

Figura 19 – Dashboard do Residente

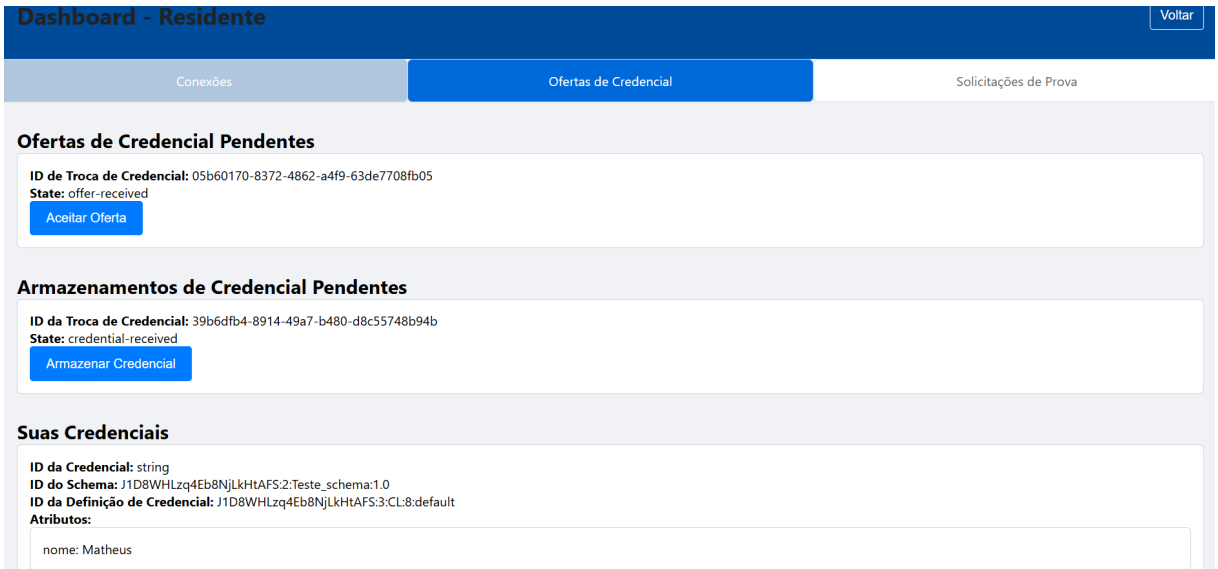


Fonte: Autoral

A Figura 19 mostra o dashboard do residente, onde o morador pode aceitar convites de conexão, apresentar provas, aceitar copiando a url enviada ou através do QR Code gerado pelo condomínio, armazenar e visualizar credenciais.

O Residente também tem a página de *Ofertas de Credencial* onde é possível visualizar as ofertas pendentes, armazenamentos pendentes e as credenciais pertencentes a ele, podemos visualizar isso na Figura 20.

Figura 20 – Ofertas de Credencial (Residente)



Fonte: Autoral

7.4.1.2 Backend

Para o nosso *backend* utilizamos o **ACA-Py** que serve como fundação para possibilitar a comunicação entre agentes. Ele é responsável por gerenciar as conexões, emissões e verificações de credenciais. E serve como uma interface para a comunicação com a blockchain, enviando e recuperando informações conforme necessário.

Figura 21 – Inicialização do agente ACA-Py

```
vscode ➜/workspaces/acapy (main) $ aca-py start    --endpoint http://localhost:8020    --label "Condominio"    --inbound-transport http 0.0.0
.0 8020    --outbound-transport http    --admin 0.0.0.0 8021    --admin-insecure-mode    --wallet-type askar    --wallet-name condominio_walle
t    --wallet-key condominio_wallet_key    --genesis-url http://localhost:9000/genesis    --auto-provision    --log-level error

::::::::::::::::::::::::::::::::::::::::::::::::
::                Condominio                  ::
::                                            ::
::                                            ::
:: Inbound Transports:                        ::
::                                            ::
::   - http://0.0.0.0:8020                    ::
::                                            ::
:: Outbound Transports:                       ::
::                                            ::
::   - http                                   ::
::   - https                                  ::
::                                            ::
:: Public DID Information:                    ::
::                                            ::
::   - DID: J1D8WHLzq4Eb8NjLkHtAFS            ::
::                                            ::
:: Administration API:                        ::
::                                            ::
::   - http://0.0.0.0:8021                    ::
::                                            ::
::                                ver: 1.1.0 ::
::::::::::::::::::::::::::::::::::::::::::::::::
```

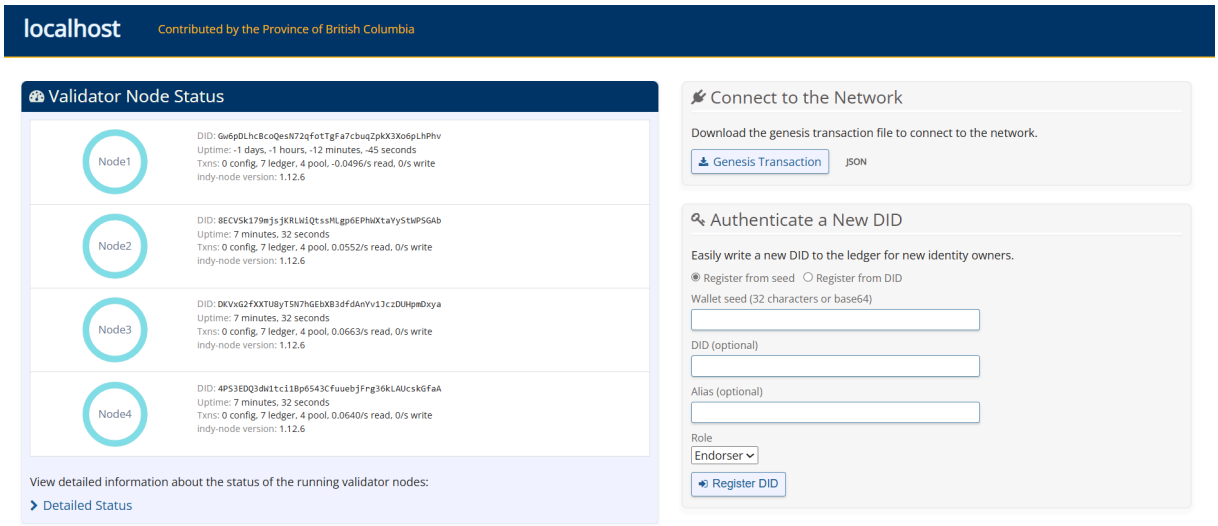
Fonte: Autoral

A Figura 21 é uma captura de tela de um terminal com a execução do comando aca-py start, que inicializa o agente, nesse caso o condomínio. Esse comando está definindo detalhes específicos sobre os transportes de entrada e saída, além disso, a configuração inclui um DID público associado ao condomínio, bem como um ponto de acesso para a API de administração no endereço especificado. O comando também especifica parâmetros como o endpoint público, o modo de execução inseguro, informações da *wallet*, detalhes sobre o arquivo de gênese da rede e configurações automáticas de provisionamento.

7.4.1.3 Blockchain

Para a *Blockchain* utilizamos a **Von-Network** para simular uma rede de nós Hyperledger Indy localmente. É aqui que os Schemas e as definições de credenciais são registrados, permitindo que os agentes possam emitir e verificar credenciais de forma segura e descentralizada. A Von-Network disponibiliza uma interface gráfica que permite a visualização dos nós e a interação dos agentes (suas DIDs) com a blockchain.

Figura 22 – Interface da Von-Network

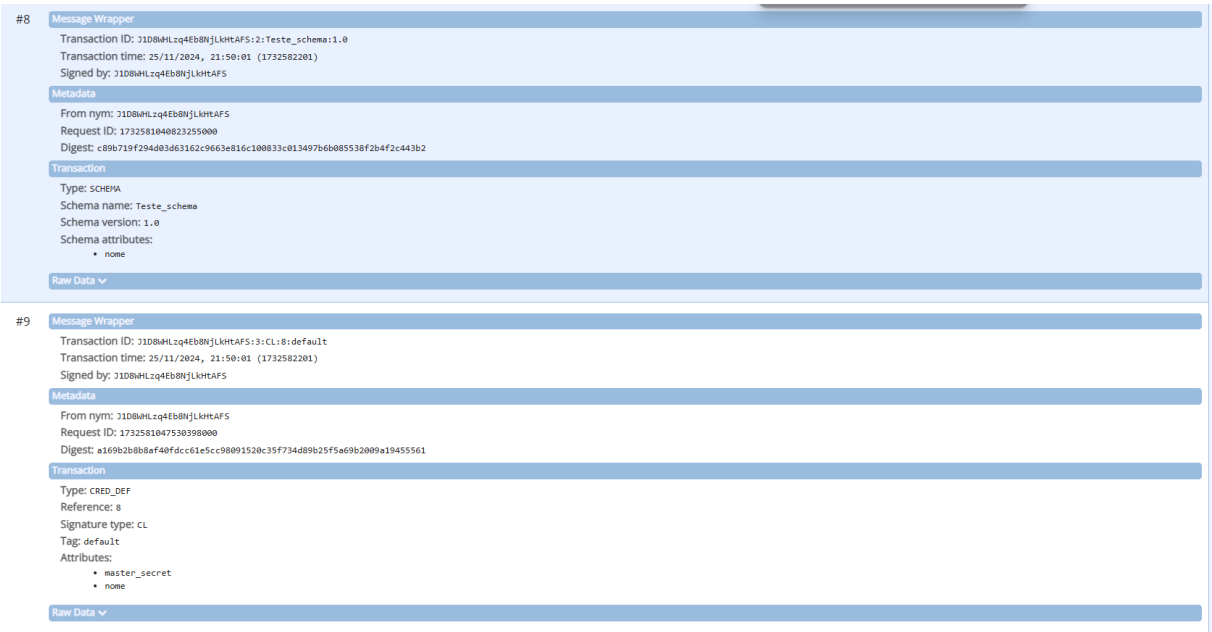


Fonte: Autoral

A Figura 22 mostra a interface da Von-Network, onde é possível visualizar os nós da rede com suas respectivas DIDs e a versão do Hyperledger Indy que cada nó está rodando. Essa interface também nos permite autenticar uma nova DID na rede de maneira manual.

A Figura 23 nos possibilita visualizar os registros dentro da blockchain, como a criação de Schemas, definições de credenciais, mostrando quem os criou (chamado de Nym), sua assinatura, quando foi criado, os identificadores e os atributos que foram definidos.

Figura 23 – Registros dentro da Blockchain



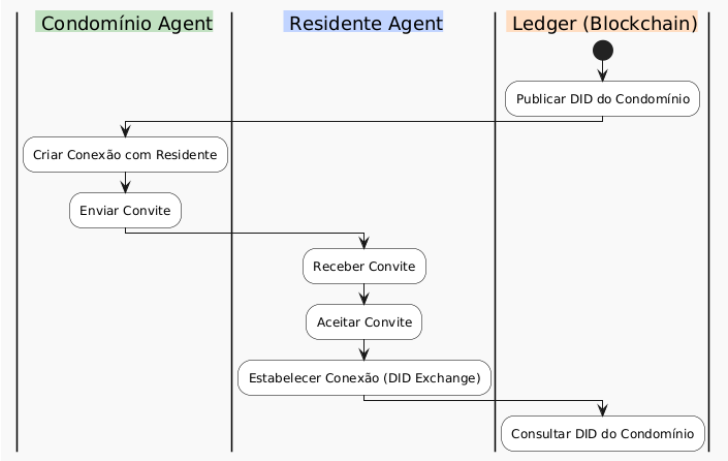
Fonte: Autoral

7.4.2 Fluxo de uso da aplicação

7.4.2.1 Fluxo da conexão entre agentes

Inicialmente para que possamos realizar a comunicação entre o Condomínio e os seus residentes uma conexão segura precisa ser estabelecida entre eles, para isso seguimos um fluxo de acordo com o diagrama a seguir:

Figura 24 – Diagrama de conexão entre agentes



Fonte: Autoral

A Figura 24 demonstra o fluxo para a realização de uma conexão segura entre dois agentes, primeiro uma DID pública do Condomínio deve ser publicada no ledger, após essa publicação o condomínio deverá criar um convite para conexão, esse convite deve ser compartilhado com o residente, que por sua vez irá aceitar esse convite assim estabelecendo a conexão.

Um exemplo de um convite de conexão gerado pelo Condomínio é mostrado na Figura 25. Onde temos o URL do convite que pode ser compartilhado com o Residente, além de um QR Code que pode ser escaneado para aceitar o convite.

Figura 25 – Convite de conexão (Condomínio)

Fonte: Autoral

Outro exemplo é a exibição no painel de uma conexão que já está ativa e outra conexão que está com o status de *request*, pois ainda não foi aceita pelo Condomínio, podemos visualizar isso na Figura 26. A interface do Residente é equivalente a essa, apenas com a diferença de que ele pode aceitar ou recusar as conexões.

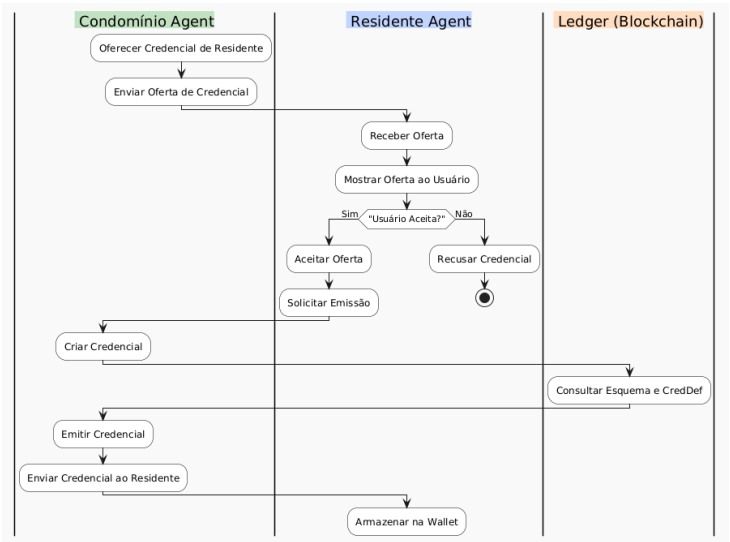
Figura 26 – Conexões (Condomínio)

Fonte: Autoral

7.4.2.2 Fluxo da emissão de credencial

Após uma conexão segura ser estabelecida, o condomínio pode emitir uma credencial para o residente, o fluxo para a emissão de uma credencial é ilustrado no diagrama a seguir:

Figura 27 – Emissão de credencial

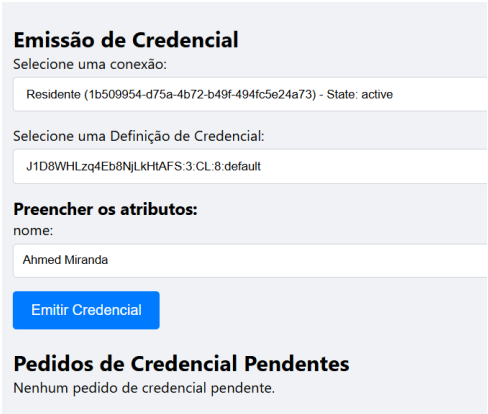


Fonte: Autoral

A Figura 27 descreve o fluxo de emissão de credenciais, do agente Condomínio para o agente Residente. O processo inicia com o Condomínio enviando uma oferta de credencial para o Residente, após o Residente aceitar a oferta e solicitar a emissão, o Condomínio emite a credencial para o Residente, que por sua vez armazena a credencial recebida.

Um exemplo do início de uma oferta de credencial pode ser visto na Figura 28, onde o Condomínio está oferecendo uma credencial para o Residente. Que por sua vez pode aceitar a oferta e armazenar a credencial como visto na Figura 20.

Figura 28 – Oferta de credencial (Condomínio)



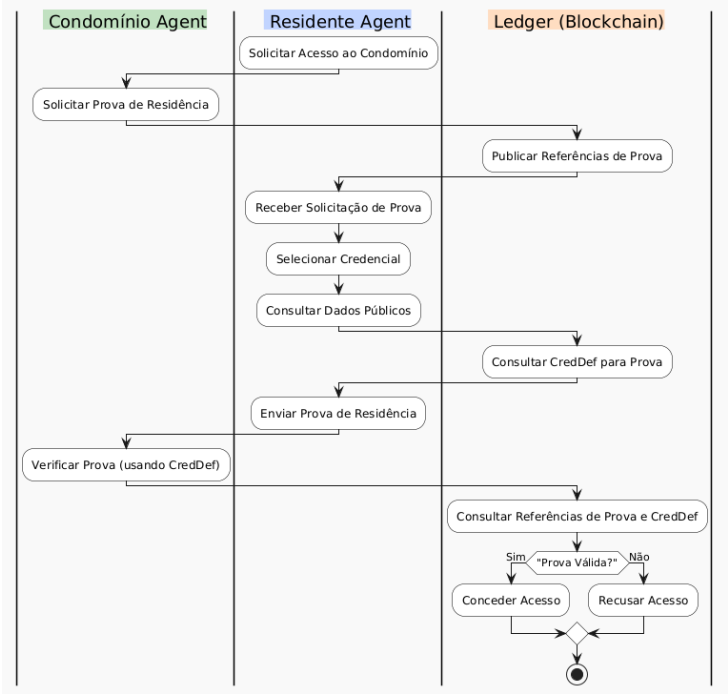
Fonte: Autoral

7.4.2.3 Fluxo da verificação de credencial

Após a emissão de uma credencial, o residente pode utilizá-la para acessar o condomínio, o fluxo para a verificação de uma credencial é ilustrado no diagrama a seguir:

A Figura 29 descreve o fluxo de verificação de credenciais, do agente Residente para o agente Condomínio. O Condomínio solicita uma prova de residência ao Residente, que por sua vez apresenta a credencial emitida pelo Condomínio. O Condomínio verifica a credencial e concede o acesso ao Residente.

Figura 29 – Verificação de credencial



Fonte: Autoral

Um exemplo do Condomínio solicitando uma prova de residência pode ser visto na Figura 30. Onde o Condomínio está solicitando uma prova de residência, baseada na definição de credencial previamente criada.

Figura 30 – Prova de residência (Condomínio)

Fonte: Autoral

Então o Residente apresenta a credencial para o Condomínio, como visto na Figura 31. Onde o Residente está apresentando a credencial para o Condomínio, que por sua vez verifica a credencial e concede o acesso ao Residente. 1

Figura 31 – Apresentar prova (Residente)

Fonte: Autoral

7.4.3 Tabelas das requisições

Para a realização de todas as interações entre os agentes que acabamos de mostrar, são utilizadas chamadas de API REST. A seguir mostramos o workflow necessário para a emissão e verificação de credenciais, com as tabelas de requisições que são feitas entre os agentes.

7.4.3.1 Tabela de emissão de credencial

As emissões de credenciais funcionam através de chamadas de API entre o *Issuer* (Emissor - Condomínio) e o *Holder* (Titular - Residente) como ilustrado na figura a seguir:

A Figura 32 descreve o fluxo de um protocolo de emissão de credenciais digitais, envolvendo dois agentes: Condomínio que representa o issuer e Residente representando o holder. Cada linha detalha uma ação sequencial no processo de emissão e armazenamento de uma credencial digital, que segue um padrão de uma API REST.

Figura 32 – *Pipeline* de emissão de credenciais

Passo do protocolo	Condomínio (Emissor)	Residente (Portador)	Observações
Envia a oferta de credencial	`POST /issue-credential-2.0/send-offer`		Serviço REST
Recebe a oferta		/issue_credential_v2_0/	callback
Envia a requisição de credencial		`POST /issue-credential-2.0/records/{cred_ex_id}/send-request`	Serviço REST
Recebe a requisição	/issue_credential_v2_0/		callback
Emite a credencial	`POST /issue-credential-2.0/records/{cred_ex_id}/issue`		Serviço REST
Recebe a credencial		/issue_credential_v2_0/	callback
Armazena a credencial		`POST /issue-credential-2.0/records/{cred_ex_id}/store`	Serviço REST
Receber uma confirmação	/issue_credential_v2_0/		callback
Armazena o id da credencial			função da aplicação

Fonte: (OpenWallet Foundation, 2024)

Abaixo estão descritos todos os passos para a realização do protocolo de emissão de credenciais:

1. **Envio de Oferta de Credencial**: Condomínio envia uma oferta de credencial através do endpoint POST /issue-credential-2.0/send-offer.

2. **Recebimento da Oferta**: Residente recebe a oferta por meio de um callback no endpoint /issue_credential_v2_0/.

3. **Envio da Solicitação de Credencial**: Residente, envia uma solicitação de credencial utilizando o endpoint POST /issue-credential-2.0/records/{cred_ex_id}/send-request.

4. **Recebimento da Solicitação**: Condomínio recebe a solicitação de credencial de Residente através de um callback no endpoint /issue_credential_v2_0/.

5. **Emissão da Credencial**: Após a solicitação de Residente, Condomínio emite a credencial usando o endpoint POST /issue-credential-2.0/records/{cred_ex_id}/issue.

6. **Recebimento da Credencial**: Residente recebe a credencial emitida via callback no endpoint `/issue_credential_v2_0/`.

7. **Armazenamento da Credencial**: Residente armazena a credencial recebida utilizando o endpoint `POST /issue-credential-2.0/records/{cred_ex_id}/store`, por meio de um serviço REST.

8. **Recebimento do Acknowledgement**: Condomínio recebe um reconhecimento de recebimento da credencial no endpoint `/issue_credential_v2_0/`, indicando que Residente armazenou a credencial.

9. **Armazenamento do ID da Credencial**: Residente armazena o identificador da credencial em sua aplicação, concluindo o processo de forma independente.

7.4.3.2 Tabela de verificação de credencial (Prova)

A verificação de credenciais funciona através de chamadas de API entre o *Verifier* (Verificador - Condomínio) e o *Prover* (Provador - Residente) como ilustrado na figura a seguir:

A Figura 33 descreve o fluxo de um protocolo de verificação de credenciais digitais, envolvendo dois agentes: Condomínio representado pelo verifier e Residente representando o prover. Cada linha detalha uma ação sequencial no processo de apresentação e verificação de uma credencial digital, que segue um padrão de uma API REST.

Figura 33 – *Pipeline* de verificação de credenciais

Passo do protocolo	Condomínio(verificador)	Residente(Portador/Provador)	Observações
Envia requisição de prova	`POST /present-proof-2.0/send-request`		serviço REST
Recebe uma Requisição de prova		/present_proof_v2_0	callback (webhook)
Encontra as credenciais		`GET /present-proof-2.0/records/{pres_ex_id}/credentials`	Serviço REST
Seleciona as credenciais			função da aplicação ou do usuário
Envia a prova		`POST /present-proof-2.0/records/{pres_ex_id}/send-presentation`	Serviço REST
Recebe a prova	/present_proof_v2_0		callback (webhook)
Valida a prova	`POST /present-proof-2.0/records/{pres_ex_id}/verify-presentation`		Serviço REST
Armazena a prova			Dados da aplicação

Fonte: (OpenWallet Foundation, 2024)

Abaixo estão descritos todos os passos para a realização do protocolo de verificação:

1. **Envio de Pedido de Prova**: Condomínio envia um pedido de prova através do endpoint POST /present-proof-2.0/send-request.

2. **Recebimento do Pedido**: Residente recebe o pedido de prova por meio de um callback no endpoint /present_proof_v2_0/.

3. **Envio da Prova**: Residente envia a prova solicitada utilizando o endpoint POST /present-proof-2.0/records/{pres_ex_id}/send-presentation.

4. **Recebimento da Prova**: Condomínio recebe a prova de Residente através de um callback no endpoint /present_proof_v2_0/.

5. **Verificação da Prova**: Condomínio verifica a prova recebida de Residente usando o endpoint POST /present-proof-2.0/records/{pres_ex_id}/verify-presentation.

6. **Recebimento do Resultado**: Residente recebe o resultado da verificação da prova via callback no endpoint /present_proof_v2_0/.

7. **Armazenamento do Resultado**: Residente armazena o resultado da verificação da prova em sua aplicação, concluindo o processo de forma independente.

8 Resultados e discussão

A implementação da prova de conceito demonstrou a viabilidade de um sistema baseado em SSI no ambiente de condomínios, utilizando tecnologias como Hyperledger Aries, ACA-Py, e blockchain Hyperledger Indy. Foi possível obter uma compreensão geral de como um esboço para uma aplicação SSI para o acesso em condomínios poderia ser desenvolvida e como os conceitos revistos nos capítulos anteriores servem para garantir a segurança e privacidade dos dados dos residentes.

Através de uma interface web conseguimos mostrar o potencial de um sistema de controle de acesso descentralizado, onde os residentes possuem controle sobre suas próprias identidades, compartilhando informações de forma segura e transparente. E onde o condomínio pode emitir e gerenciar as credenciais dos moradores de forma eficiente e segura.

Não podemos deixar de citar também que a comunidade envolvida nos projetos foi muito atenciosa e prestativa em momentos de dificuldade para entender os conceitos de SSI e como poderíamos aplicá-los em nossa prova de conceito.

Dito isso, acreditamos que os resultados obtidos com a implementação da prova de conceito são promissores e que a plataforma utilizada para o desenvolvimento é robusta e confiável, mesmo estando em seus estágios iniciais de desenvolvimento. Podendo ser expandida e aprimorada para o atendimento das necessidades de um condomínio real.

9 Conclusão

Esta monografia apresenta a construção de uma prova de conceito onde foi desenvolvido um sistema descentralizado de gerenciamento de identidades baseado em SSIs para o contexto de autenticação de residentes de um condomínio, com o objetivo de avaliar a viabilidade desses sistemas e identificar seus prós e contras.

No desenvolvimento desse trabalho encontramos uma série de desafios, o conceito de Identificadores descentralizados e de identidades auto soberanas é relativamente novo, assim como as tecnologias e frameworks necessários para o seu desenvolvimento. Constantemente eles são atualizados tendo suas versões antigas depreciadas assim necessitando de constantes adaptações às mudanças tecnológicas.

Apesar desses desafios, o modelo SSI mostrou-se viável e eficiente. A abordagem descentralizada, com o uso de credenciais verificáveis, garantiu que os moradores tivessem controle sobre suas informações, evitando a dependência de intermediários e minimizando o risco de vazamento de dados, como os registrados em sistemas centralizados.

Por fim, este estudo reafirma a relevância e o potencial transformador das identidades digitais auto-soberanas, não apenas em condomínios, mas em diversas outras aplicações. Os avanços alcançados indicam que futuras pesquisas devem explorar maneiras de superar as limitações técnicas e integrar SSIs a outros sistemas e contextos, contribuindo para a evolução de um ecossistema digital mais seguro e centrado no usuário.

9.1 Trabalhos futuros

A aplicação desenvolvida mostrou-se um protótipo funcional que pode ser utilizado como base para futuras implementações e melhorias. Com a evolução constante das tecnologias SSI é natural que o processo de implementação dessas melhorias seja facilitado e que novas funcionalidades possam ser adicionadas ao sistema.

Dito isso, uma possível melhoria é a separação do agente Condomínio em dois agentes distintos, o Condomínio continua como ***verificador*** para realizar a verificação das credenciais dos residentes e um novo agente, a administradora do condomínio, que é o ***emissor*** das credenciais para os moradores. Essa separação reflete a realidade de muitos condomínios, onde a administradora é responsável por cuidar dos moradores e suas informações, enquanto o condomínio é responsável pela segurança e controle de acesso.

Outra possível melhoria é o desenvolvimento de carteiras mobile para os moradores, visando facilitar ainda mais a interação com o sistema. Essas carteiras funcionariam como aplicativos instalados nos dispositivos móveis dos usuários, permitindo que gerenciem suas credenciais digitais de forma segura e prática.

Visando uma melhoria na eficiência e usabilidade do sistema de controle de acesso, um direcionamento para trabalhos futuros é a substituição da leitura de QR Codes pelo uso de tecnologia NFC (Near Field Communication) para realizar a conexão entre os agentes. Essa abordagem complementa a proposta da carteira mobile, dessa maneira o residente poderia apenas aproximar o dispositivo móvel com a credencial armazenada do leitor NFC para liberar o acesso.

Referências

Access Run. **Gestão de Acesso e Identidades: o que é e como ela pode ser útil em sua empresa e condomínio**. 2019. Disponível em: <www.access.run/2019/06/gestao-de-acesso-e-identidades-o-que-e-e-como-ela-pode-ser-util-em-sua-empresa-e-condominio>. Citado na página 21.

ALLEN, C. The path to self-sovereign identity. **Life With Alacrity**, 2017. Disponível em: <https://www.lifewithalacrity.com/article/the-path-to-self-soverereign-identity/>. Citado 2 vezes nas páginas 32 e 42.

ALZAHRANI; BANDER. An information-centric networking based registry for decentralized identifiers and verifiable credentials. **IEEE Access**, v. 8, p. 137198–137208, 2020. Citado na página 21.

ANDERSON, W. **DIDs On The Blockchain: How Decentralized Identifiers Are The Future Of Online Identity**. [S.l.], 2023. Disponível em: <https://www.forbes.com/sites/williamanderson/2023/03/15/dids-on-the-blockchain-how-decentralized-identifiers-are-the-future-of-online-identity/?sh=55ade2ea138d>. Citado na página 37.

BCgov. **von-network**. 2017. Disponível em: <https://github.com/bcgov/von-network/tree/main>. Citado na página 56.

BERNERS-LEE, T.; FIELDING, R. T.; MASINTER, L. M. **Uniform Resource Identifier (URI): Generic Syntax**. RFC Editor, 2005. RFC 3986. (Request for Comments, 3986). Disponível em: <https://www.rfc-editor.org/info/rfc3986>. Citado 2 vezes nas páginas 37 e 38.

Bit2Me Academy. **Hash function: concept and application in Bitcoin**. 2018. Acesso em: 03 out. 2024. Disponível em: <https://academy.bit2me.com/wp-content/uploads/2018/10/funcion_hash_criptomonedas.webp>. Citado na página 26.

BRANDS, S. Restrictive binding of secret-key certificates. In: **Proceedings of the 14th Annual International Conference on Theory and Application of Cryptographic Techniques**. Berlin, Heidelberg: Springer-Verlag, 1995. (EUROCRYPT'95), p. 231–247. ISBN 3540594094. Citado na página 45.

BUTTERWORTH, T.; CURREN, S. **Trust, Verifiable Credentials, and Interoperability**. [S.l.], 2022. Disponível em: <https://indicio.tech/wp-content/uploads/2022/04/Indicio_Report_TrustVerifiableCredentialsInteroperability_040622.pdf>. Citado na página 42.

CHADWICK, D. W. Federated identity management. In: _____. **Foundations of Security Analysis and Design V: FOSAD 2007/2008/2009 Tutorial Lectures**. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. p. 96–120. ISBN 978-3-642-03829-7. Disponível em: <https://doi.org/10.1007/978-3-642-03829-7_3>. Citado na página 31.

CHAUM, D. Security without identification: transaction systems to make big brother obsolete. **Commun. ACM**, Association for Computing Machinery, New York, NY, USA, v. 28, n. 10, p. 1030–1044, out. 1985. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/4372.4373>. Citado na página 44.

DIFFIE, W.; HELLMAN, M. E. New directions in cryptography. **IEEE Transactions on Information Theory**, v. 22, n. 6, p. 644–654, November 1976. Citado na página 25.

Docker Inc. **Docker**. 2023. Disponível em: <https://www.docker.com/>. Citado na página 55.

Federal Trade Commission. **Equifax Data Breach Settlement**. 2024. Acesso em: 03 out. 2024. Disponível em: <https://www.ftc.gov/enforcement/refunds/equifax-data-breach-settlement>. Citado na página 31.

FOUNDATION, H. **LF Decentralized Trust**. 2024. Disponível em: <https://lf-hyperledger.atlassian.net/wiki/spaces/HYP/overview?mode=global>. Citado na página 47.

FOUNDATION, L. **Hyperledger Anoncreds**. 2024. Disponível em: <https://www.lfdecentralizedtrust.org/projects/anoncreds>. Citado na página 48.

_____. **Hyperledger Aries**. 2024. Disponível em: <https://www.lfdecentralizedtrust.org/projects/aries>. Citado na página 48.

_____. **Hyperledger Fabric**. 2024. Disponível em: <https://www.lfdecentralizedtrust.org/projects/fabric>. Citado na página 48.

_____. **Hyperledger Indy**. 2024. Disponível em: <https://www.lfdecentralizedtrust.org/projects/hyperledger-indy>. Citado 2 vezes nas páginas 22 e 48.

G1. **Polícia Civil investiga se ataque a sistema de reconhecimento facial expôs dados de moradores de condomínios no interior de SP**. 2024. Acessado em: 24 nov. 2024. Disponível em: <https://g1.globo.com/sp/sorocaba-jundiai/noticia/2024/05/05/policia-civil-investiga-se-ataque-a-sistema-de-reconhecimento-facial-expos-dados-de-moradores-de-condomin ghtml>. Citado na página 51.

GURURAJ, P. Identity management using permissioned blockchain. p. 1–3, 2020. Citado na página 36.

HERBKE, P.; SAPKOTA, A.; LAMICHHANE, S. **Lifecycle Management of Resumés with Decentralized Identifiers and Verifiable Credentials**. 2024. Disponível em: <https://arxiv.org/abs/2406.11535>. Citado na página 43.

Hyperledger Foundation. **Aries Cloud Agent Python (ACA-Py)**. 2024. Disponível em: <https://github.com/openwallet-foundation/acapy>. Citado 2 vezes nas páginas 53 e 54.

KATZ, J.; LINDELL, Y. **Introduction to Modern Cryptography**. CRC Press, 2020. (Chapman & Hall/CRC Cryptography and Network Security Series). ISBN 9781351133012. Disponível em: <https://books.google.com.br/books?id=RsoOEAAAQBAJ>. Citado na página 25.

KONGSUWAN, N.; TOSIRISUK, R. Anonymous credential part 1: Brief overview and history. **Finema on Medium**, 2020. Disponível em: <https://medium.com/finema/anonymous-credential-part-1-brief-overview-and-history-c6679034c914>. Citado na página 46.

KORTESNIEMI, Y.; LAGUTIN, D.; ELO, T.; FOTIOU, N. Improving the privacy of iot with decentralised identifiers (dids). **Journal of Computer Networks and Communications**, 2019. Disponível em: <https://www.hindawi.com/journals/jcnc/2019/8706760/>. Citado na página 37.

Linux Foundation. **Linux Foundation Unites Industry Leaders to Advance Blockchain Technology**. 2015. Acessado em: 11 de novembro de 2024. Disponível em: <https://www.lfdecentralizedtrust.org/announcements/2015/12/17/linux-foundation-unites-industry-leaders-to-advance-blockchain-technology>. Citado na página 47.

_____. **indy-node-docs**. 2024. Disponível em: <https://hyperledger-indy.readthedocs.io/projects/node/en/latest/index.html>. Citado na página 56.

MARTINOVIC LUCAS KELLO, I. S. I. Blockchains for governmental services: Design principles, applications, and case studies. 12 2017. Disponível em: <https://www.politics.ox.ac.uk/sites/default/files/2022-03/201712-CTGA-Martinovic%20I-Kello%20L-blockchainsforgovernmentalservices.pdf>. Citado na página 36.

Meta Platforms, Inc. **React.js**. 2023. Disponível em: <https://reactjs.org/>. Citado na página 55.

MIT-IDE. **Digital Identity: The Key to Privacy and Security in the Digital World**. 2021. Acesso em: 03 out. 2024. Disponível em: <https://ide.mit.edu/insights/digital-identity-the-key-to-privacy-and-security-in-the-digital-world/>. Citado na página 29.

NAIK, N.; JENKINS, P. uport open-source identity management system: An assessment of self-sovereign identity and user-centric data platform built on blockchain. p. 1–7, 2020. Citado na página 49.

NAKAMOTO, S. **Bitcoin: A Peer-to-Peer Electronic Cash System**. 2008. Acesso em: 03 out. 2024. Disponível em: <https://bitcoin.org/bitcoin.pdf>. Citado na página 35.

Okta. **Self-Sovereign Identity**. 2024. Disponível em: <https://www.okta.com/identity-101/self-sovereign-identity/>. Citado 2 vezes nas páginas 21 e 37.

Omar Dib, Khalifa Toumi. Decentralized identity systems: Architecture, challenges, solutions and future directions. 2020. Citado na página 31.

OpenAPI Initiative. **OpenAPI Specification**. 2024. Disponível em: <https://www.openapis.org/>. Citado na página 55.

OpenWallet Foundation. **openAPI - ACA-Py Docs**. [S.l.], 2024. Acessado em: 10 de novembro de 2024. Disponível em: <https://aca-py.org/main/demo/OpenAPIDemo/>. Citado 3 vezes nas páginas 55, 67 e 69.

ORACLE. **What is Data Security?** 2024. Acesso em: 17 set. 2024. Disponível em: <https://www.oracle.com/br/security/database-security/what-is-data-security/>. Citado na página 24.

Python Software Foundation. **Python Programming Language**. 2023. Disponível em: <https://www.python.org/>. Citado na página 53.

RIVEST, R. L.; SHAMIR, A.; ADLEMAN, L. A method for obtaining digital signatures and public-key cryptosystems. **Commun. ACM**, Association for Computing Machinery, New York, NY, USA, v. 21, n. 2, p. 120–126, fev. 1978. ISSN 0001-0782. Disponível em: <https://doi.org/10.1145/359340.359342>. Citado na página 27.

RODRIGUES, H. **(in)Seguranca na Prescrição Eletrónica**. Tese (Doutorado), 10 2013. Citado na página 27.

SOVRIN. **Sovrin: A Protocol and Token for Self-Sovereign Identity and Decentralized Trust**. 2018. Versão 1.0. Disponível em: <https://sovrin.org/library/sovrin-protocol-and-token-white-paper/>. Citado na página 48.

SPORNY, M.; LONGLEY, D.; CHADWICK, D. **Verifiable Credentials Data Model v1.1**. 2022. Disponível em: <https://www.w3.org/TR/vc-data-model/>. Citado 4 vezes nas páginas 41, 42, 43 e 44.

SPORNY, M.; LONGLEY, D.; SABADELLO, M.; REED, D.; STEELE, O.; ALLEN, C. Decentralized identifiers (dids) v1.0. **W3C**, 2022. Disponível em: <https://www.w3.org/TR/did-core/>. Citado 5 vezes nas páginas 21, 37, 38, 39 e 40.

STALLINGS, W. **Cryptography and Network Security: Principles and Practice, Global Edition**. Pearson Education, 2022. ISBN 9781292437491. Disponível em: <https://books.google.com.br/books?id=E7-GEAAAQBAJ>. Citado 2 vezes nas páginas 24 e 26.

STOKKINK, Q.; POUWELSE, J. Deployment of a blockchain-based self-sovereign identity. p. 1336–1342, 2018. Citado na página 32.

UFRJ, G. de Teleinformática e A. G. **Imagem ilustrativa de blocos em blockchain**. 2019. Disponível em: <https://www.gta.ufrj.br/ensino/eel878/redes1-2019-1/v1/blockchain/blocks.png>. Citado na página 36.

UNIVERSITY, D. **5 Ways Decentralized Identity is a THE FUTURE for Web 3.0**. 2022. <https://www.youtube.com/watch?v=iaMi8gRFHc4&ab_channel=DappUniversity>. Citado na página 37.

World Economic Forum. **A Blueprint for Digital Identity**. [S.l.], 2016. Acesso em: 03 out. 2024. Disponível em: <https://www3.weforum.org/docs/WEF_A_Blueprint_for_Digital_Identity.pdf>. Citado na página 30.

_____. **Digital Identity: On the Threshold of a Digital Identity Revolution**. [S.l.], 2018. Disponível em: <https://www3.weforum.org/docs/White_Paper_Digital_Identity_Threshold_Digital_Identity_Revolution_report_2018.pdf>. Citado na página 21.

YANG, X.; LI, W. A zero-knowledge-proof-based digital identity management scheme in blockchain. **Computers Security**, v. 99, p. 102050, 2020. ISSN 0167-4048. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0167404820303230>. Citado na página 37.

ZHOU, L.; DIRO, A.; SAINI, A.; KAISAR, S.; HIEP, P. C. Leveraging zero knowledge proofs for blockchain-based identity sharing: A survey of advancements, challenges and opportunities. **Journal of Information Security and Applications**, v. 80, p. 103678, 2024. ISSN 2214-2126. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2214212623002624>. Citado na página 45.

ZYSKIND, G.; NATHAN, O.; PENTLAND, A. S. Decentralizing privacy: Using blockchain to protect personal data. In: **2015 IEEE Security and Privacy Workshops**. [s.n.], 2015. p. 180–184. Acesso em: 03 out. 2024. Disponível em: <https://ieeexplore.ieee.org/ielx7/7160794/7163193/07163223.pdf>. Citado 2 vezes nas páginas 35 e 36.

Apêndices

APÊNDICE A – Preparação do ambiente

A.1 Pré requisitos

- VS code instalado.
- Extensão "Remote - Containers"instalada no VS Code.
- Docker desktop instalado.

Após garantir a instalação desses três pré requisitos execute o seguinte comando no seu terminal:

Listing A.1 – Comando para clonar o repositório com submódulos

```
git clone --recurse-submodules
https://github.com/mirandamath/SSI-PoC-TCC.git
```

Agora com o repositório clonado podemos começar a rodar a aplicação, basta abrir o VS code na pasta **acapy** do repositório, e rodar o comando **rebuildAndReopenInContainer** da extensão **Remote-containers**, agora estamos prontos para iniciar a nossa aplicação.

APÊNDICE B – Executação do sistema

B.1 Frontend

Após abrirmos o VS code na pasta **acapy** do repositório e estarmos dentro do contêiner, abra Quatro terminais, em um deles execute os seguintes comandos:

Listing B.1 – Mudando para o diretório do frontend

```
cd ../frontend
```

Listing B.2 – Inicializando o frontend

```
yarn start
```

B.2 Von network

Com esses comandos o *frontend* da nossa aplicação foi inicializado, agora, no segundo terminal, execute os comandos a seguir:

Listing B.3 – Mudando para o diretório do ledger

```
cd ../von-network
```

```
./manage start
```

B.3 Inicialização dos agentes

Agora podemos inicializar os dois agentes. Inicialize dois terminais, ambos dentro da pasta **acapy**. Em um inicializaremos o Condomínio e no outro o Residente.

Listing B.4 – Comando para inicialização do residente

```
aca-py start \
  --endpoint http://localhost:8030/ \
  --label "Residente" \
  --inbound-transport http 0.0.0.0 8030 \
  --outbound-transport http \
  --admin 0.0.0.0 8031 \
  --admin-insecure-mode \
  --wallet-type askar \
  --wallet-name residente_wallet \
  --wallet-key residente_wallet_key \
  --genesis-url http://localhost:9000/genesis \
  --auto-provision
```

Listing B.5 – Comando para inicialização do condomínio

```
aca-py start \
  --endpoint http://localhost:8020/ \
  --label "Condominio" \
  --inbound-transport http 0.0.0.0 8020 \
  --outbound-transport http \
  --admin 0.0.0.0 8021 \
  --admin-insecure-mode \
  --wallet-type askar \
  --wallet-name condominio_wallet \
  --wallet-key condominio_wallet_key \
  --genesis-url http://localhost:9000/genesis \
  --auto-provision
```

Após inicializarmos o condomínio precisamos deixar a sua DID pública no ledger. Para isso entre na API do agente condomínio localizada no endereço http://localhost:8021 e vá até o endpoint POST /wallet/did/create nele clique em *try it out* e copie o json a seguir no body e depois execute.

Listing B.6 – JSON para criação de DID local

```
{
  "method": "sov",
  "options": {
    "key_type": "ed25519"
  }
}
```

Após executar o endpoint anterior copie os elementos did e verkey do *Response Body* e vá até o ledger, acessado atavés do endereço http://localhost:9000. Na área denominada de *Authenticate a New DID* selecione a opção *Register from DID* e preencha os campos com as informações que acabamos de copiar e clique no botão *Register DID*. Volte para a API e no endpoint POST /wallet/did/public copie a did que acabamos de registrar no ledger, no campo did* e execute.